News Service

Documentación técnica completa

Generado: 08/06/2026

Overview

The **Althara News Service** is a specialized news microservice designed to automate the ingestion, processing, and distribution of industry-specific content for two distinct brands: **Althara** (Real Estate) and **Oxono** (Tech/AI). Built with FastAPI, it provides a full pipeline from raw RSS feeds to structured social media drafts, managed through an internal "News Studio" interface.

README.md:1-4, app/main.py:11-11

System Purpose and Brands

The service operates on a dual-brand architecture, segregating content by domain while sharing a unified core for ingestion and transformation.

- **Althara (Real Estate):** Focuses on Spanish and international real estate markets, economy, and urban development.
- **Oxono (Tech/AI):** Focuses on technology trends, artificial intelligence, and digital innovation.

app/brands.py:1-15, README.md:32-35

The News Pipeline

The system follows a linear progression from external data sources to actionable social media content.

High-Level Pipeline Flow

The following diagram illustrates the data flow from ingestion to the News Studio UI.

```
graph TD
    subgraph "External Sources"
        RSS["RSS Feeds"]
        IDL["Idealista API"]
    end
```

```
    subgraph "Ingestion Space (app/ingestion/)"
        RI["rss_ingestor.py"]
        TI["tech_rss_ingestor.py"]
        II["idealista_ingestor.py"]
        GR["guardrails.py"]
    end

    subgraph "Storage & Logic"
        DB[("Neon PostgreSQL (app/database.py)")]
        NA["news_adapter.py"]
        IA["ig_adapter.py"]
    end

    subgraph "Presentation"
        UI["News Studio (app/routers/ui.py)"]
        API["FastAPI Docs (/docs)"]
    end

    RSS --> RI & TI
    IDL --> II
    RI & TI & II --> GR
    GR -->|Persist News| DB
    DB --> NA -->|Structured Content| DB
    DB --> IA -->|IG Drafts| DB
    DB --> UI & API
```

Sources: app/ingestion/rss_ingestor.py:1-20, app/adapters/news_adapter.py:1-15, app/routers/ui.py:1-20

Code Entity Mapping

To bridge the gap between functional requirements and the codebase, the following diagrams map logical concepts to specific classes and files.

Ingestion and Guardrails Entity Map

This diagram shows how ingestion logic is partitioned between brands and the shared validation logic.

```
graph LR
    subgraph "Real Estate (Althara)"
        A_INGEST["rss_ingestor.py"]
```

```
        A_CONST["constants.py"]
    end

    subgraph "Tech (Oxono)"
        T_INGEST["tech_rss_ingestor.py"]
        T_CONST["constants_tech.py"]
    end

    subgraph "Shared Validation"
        PASSES["passes_guardrails()"]
        DENY["DENY_KEYWORDS"]
    end

    A_INGEST --> PASSES
    T_INGEST --> PASSES
    A_CONST -.->|Config| A_INGEST
    T_CONST -.->|Config| T_INGEST
    PASSES -->|Uses| DENY
```

Sources: app/ingestion/rss_ingestor.py:345-360, app/ingestion/tech_rss_ingestor.py:200-215, app/ingestion/guardrails.py:10-30

Content Transformation Entity Map

This diagram maps the transformation from a raw `News` record to a brand-aligned `IGDraft`.

```
graph TD
    NEWS["News Model (app/models.py)"]

    subgraph "Adapters"
        ADAPTER["NewsAdapter (news_adapter.py)"]
        IG_GEN["IGAdapter (ig_adapter.py)"]
    end

    subgraph "Structured Fields"
        STR_CONT["althara_content (JSONB)"]
        IG_POST["instagram_post (Text)"]
    end

    DRAFT["IGDraft Model (app/models.py)"]

    NEWS -->|Input| ADAPTER
    ADAPTER -->|Updates| STR_CONT
    ADAPTER -->|Updates| IG_POST
```

```
    NEWS -->|Input| IG_GEN
    IG_GEN -->|Creates| DRAFT
```

Sources: app/models.py:12-65, app/adapters/news_adapter.py:40-60, app/adapters/ig_adapter.py:15-40

Key Subsections

Getting Started

Detailed instructions on setting up the local environment, including Python 3.11 requirements, environment variables for the Neon PostgreSQL connection, and running Alembic migrations. For details, see Getting Started.

Project Structure

A walkthrough of the directory layout, explaining the separation between the FastAPI application (`app/`), database migrations (`alembic/`), and maintenance utilities (`scripts/`). For details, see Project Structure.

Core Architecture

Deep dive into the system's foundation: the FastAPI bootstrap, the async SQLAlchemy database layer, and the brand/domain models that drive the multi-tenant logic. For details, see Core Architecture.

News Ingestion Pipeline

Explains how content is pulled from RSS feeds and APIs, filtered through keyword-based guardrails, and categorized using priority-based scoring. For details, see News Ingestion Pipeline.

Content Adaptation

Details the "Althara Adapter" and "Instagram Draft Generator" logic, which uses structured JSON schemas to turn news articles into professional summaries and social media carousels. For details, see Content Adaptation and Transformation.

API and News Studio

Reference for the REST API endpoints and the Jinja2-based News Studio UI used for reviewing and publishing content. For details, see API Reference and News Studio UI.

Data Models

Documentation of the SQLAlchemy ORM models (`News`, `IGDraft`) and the evolution of the database schema via Alembic. For details, see Data Models and Database Schema.

Getting Started

This page provides a technical guide for setting up the **althara-news-service** development environment. It covers the transition from a clean system to a fully operational FastAPI server with a synchronized database and accessible News Studio UI.

Environment Prerequisites

The service is built to run on **Python 3.11.9** runtime.txt:1-1. It relies on a PostgreSQL backend (specifically optimized for Neon) and uses `asyncio` for non-blocking database operations via `asyncpg`.

Dependency Overview

The project manages its dependencies via `requirements.txt`. Key components include: * **Web Framework**: `fastapi` and `uvicorn` requirements.txt:1-2. * **ORM & Migrations**: `sqlalchemy` (with asyncio support) and `alembic` requirements.txt:3-5. * **Data Ingestion**: `feedparser`, `httpx`, and `beautifulsoup4` requirements.txt:9-11. * **Configuration**: `pydantic-settings` for environment variable management requirements.txt:8.

Sources: requirements.txt:1-16, runtime.txt:1-1, README.md:132-137

Local Setup Workflow

To initialize the project locally, follow the sequence below to ensure the database schema and environment variables are correctly aligned.

1. Virtual Environment and Installation

```
# Create and activate virtual environment
python3.11 -m venv venv
```

```
source venv/bin/activate   # Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
```

2. Environment Variables (`.env`)

Create a `.env` file in the root directory. The application uses `pydantic-settings` to load these values.

Variable	Requirement	Description
`DATABASE_URL`	**Required**	Must use the `postgresql+asyncpg://` scheme. README.md:165-166
`INGEST_TOKEN`	Optional	Bearer token for authenticating admin ingestion endpoints.
`UI_USER`	Optional	Username for News Studio Basic Auth.
`UI_PASS`	Optional	Password for News Studio Basic Auth.

Database URL Normalization: The system automatically converts standard `postgresql://` URLs (like those copied from the Neon dashboard) to `postgresql+asyncpg://` and strips incompatible SSL parameters to ensure compatibility with the `asyncpg` driver README.md:168-172.

3. Database Migrations

The project uses Alembic for asynchronous migrations. The configuration in `alembic.ini` points to the `alembic/` directory alembic.ini:3-5, and `env.py` is configured to use an async engine README.md:43-43.

```
# Apply all migrations to the database
alembic upgrade head
```

Sources: README.md:108-124, README.md:155-180, alembic.ini:1-13

Starting the Service

Once the database is migrated, start the FastAPI server using `uvicorn`.

```
uvicorn app.main:app --reload
```

Access Points

- **REST API**: `http://localhost:8000/api`
- **Interactive API Docs (Swagger)**: `http://localhost:8000/docs` README.md: 237-237
- **News Studio UI**: `http://localhost:8000/ui` README.md:128-128

Sources: README.md:122-128, README.md:199-210

System Initialization Logic

The following diagram illustrates the relationship between the environment configuration, the Python runtime entities, and the resulting service availability.

Setup and Initialization Flow

```
graph TD
    subgraph ["Environment Space"]
        ENV[".env file"]
        DB_URL["DATABASE_URL"]
        RT["runtime.txt (Python 3.11.9)"]
    end

    subgraph ["Code Entity Space"]
        CFG["app.config.Settings"]
        ALM["alembic.env:run_migrations_online"]
        APP["app.main:app (FastAPI)"]
        DB_ENG["app.database:engine (AsyncEngine)"]
    end

    ENV --> CFG
    DB_URL --> CFG
```

```
    RT --> APP
    CFG --> DB_ENG
    DB_ENG --> ALM
    ALM --> |"Schema Creation"| DB[("Neon PostgreSQL")]
    APP --> |"Dependency Injection"| DB_ENG
```

Sources: app/config.py:1-20, app/database.py:1-15, app/main.py:1-10, README.md:155-180

Deployment Configuration

The service is pre-configured for deployment on **Render.com** using the `render.yaml` specification.

Render Build & Start Pipeline

The deployment process automates the environment setup and migration execution.

```
graph LR
    subgraph ["Build Phase"]
        B1["pip install --upgrade pip"]
        B2["pip install -r requirements.txt"]
        B3["alembic upgrade head"]
    end

    subgraph ["Runtime Phase"]
        S1["uvicorn app.main:app"]
        S2["PORT environment variable"]
    end

    B1 --> B2
    B2 --> B3
    B3 --> S1
    S2 -.-> |"Injected into"| S1
```

Deployment Commands

- **Build Command**: `pip install --upgrade pip && pip install -r requirements.txt && alembic upgrade head` render.yaml:5-5

- **Start Command**: `uvicorn app.main:app --host 0.0.0.0 --port $PORT` render.yaml:6-6

Sources: render.yaml:1-10, build.sh:1-16, README.md:181-195

Project Structure

This page provides a technical walkthrough of the `news-service` codebase directory layout. It describes how the application is organized, the purpose of each top-level directory, and how data flows between the various modules.

High-Level Directory Overview

The project follows a standard FastAPI structure, separating the core application logic from database migrations, utility scripts, and testing suites.

Directory	Purpose
`app/`	The core FastAPI application, containing business logic, models, and routes.
`alembic/`	Database migration scripts and configuration using SQLAlchemy/Alembic.
`scripts/`	Standalone Python and Shell scripts for maintenance and automation.
`tests/`	Pytest suite for unit and integration testing.
`static/`	Frontend assets (CSS, JS) for the News Studio UI.

The `app/` Directory

The `app/` directory is the heart of the service. It is structured into functional modules that separate concerns between data ingestion, content transformation, and API delivery.

Core Modules and Initialization

- `main.py` : The entry point of the application. It initializes the `FastAPI` instance app/main.py:11, configures CORS app/main.py:14-20, attaches

`RateLimitMiddleware` app/main.py:23, and registers all API and UI routers app/main.py:25-32.

- **`config.py`**: Defines the `Settings` class using Pydantic, which manages environment variables like `DATABASE_URL` and ingestion limits app/config.py:10-45.
- **`database.py`**: Manages the SQLAlchemy async engine and session factory.
- **`models/`**: Contains SQLAlchemy ORM models (e.g., `News`, `IGDraft`) that define the database schema.

Feature Modules

- **`ingestion/`**: Contains logic for fetching data from external sources. It includes specific ingestors for RSS feeds and the Idealista API app/ingestion/**init**.py:1-7.
- **`adapters/`**: Responsible for transforming raw news into brand-aligned content. This includes the Althara narrative reconstruction and Instagram draft generation app/adapters/**init**.py:1-3.
- **`routers/`**: Grouped API endpoints.
  - `news.py`: Public-facing news consumption.
  - `admin.py` / `tech_admin.py`: Ingestion and maintenance triggers.
  - `ig_drafts.py`: Management of social media drafts.
  - `ui.py`: Routes serving the Jinja2 templates for the News Studio.

Data Flow Diagram

The following diagram illustrates how a piece of news moves from an external source through the system to the database and eventually the UI.

Content Pipeline Flow

```
graph TD
    subgraph "External World"
        RSS["RSS Sources"]
        IDL["Idealista API"]
    end

    subgraph "app/ingestion"
        RI["rss_ingestor.py"]
        II["idealista_ingestor.py"]
    end

    subgraph "app/adapters"
```

```
        NA["news_adapter.py"]
        IGA["ig_adapter.py"]
    end

    subgraph "Storage & Delivery"
        DB[("PostgreSQL DB")]
        API["app/routers/"]
        UI["News Studio UI"]
    end

    RSS --> RI
    IDL --> II
    RI --> NA
    II --> NA
    NA --> IGA
    IGA --> DB
    DB --> API
    API --> UI
```

Sources: app/main.py:1-47, app/ingestion/**init**.py:1-16, app/adapters/**init**.py:1-12

Infrastructure and Automation

`alembic/`

This directory manages the evolution of the database schema. * `env.py` : Configures the migration environment to use the async engine from the `app` module. * `versions/` : Contains the individual migration scripts (e.g., adding the `domain` or `althara_content` fields). * `script.py.mako` : The template used for generating new migration files alembic/script.py.mako:1-26.

`scripts/`

Standalone utilities used for maintenance and scheduled tasks. * **Ingestion Wrappers**: Scripts like `ingest_news.py` allow running the pipeline via cron jobs without going through the HTTP layer. * **Cleanup**: `remove_irrelevant_news.py` uses the system's guardrails to prune the database.

`tests/`

The testing suite ensures the reliability of the transformation logic. * `conftest.py`: Provides shared fixtures like `sample_tech_news`. * `test_ig_adapter.py`: Validates the complex logic of Instagram carousel generation.

Code Entity Mapping

This section bridges the conceptual folders to the specific code entities defined within them.

Module to Class/Function Mapping

```
classDiagram
    class FastAPI_App {
        <<app/main.py>>
        +app: FastAPI
        +startup()
        +shutdown()
    }
    class Settings {
        <<app/config.py>>
        +DATABASE_URL: str
        +INGEST_TOKEN: str
        +REAL_ESTATE_RSS_LIMIT: int
    }
    class Ingestion_Module {
        <<app/ingestion/>>
        +RSSIngestor
        +IdealistaClient
    }
    class Adapters_Module {
        <<app/adapters/>>
        +build_althara_summary()
        +generate_variants()
    }

    FastAPI_App --> Settings : loads
    FastAPI_App --> Ingestion_Module : triggers via routers
    Ingestion_Module --> Adapters_Module : passes raw news
```

Sources: app/main.py:11-45, app/config.py:10-37, app/ingestion/**init**.py:1-7, app/adapters/**init**.py:1-3

Core Architecture

The Althara News Service is built on a layered architecture designed to ingest, process, and distribute news across two distinct domains: Real Estate (**Althara**) and Technology (**Oxono**). The system utilizes FastAPI for its web layer, SQLAlchemy for asynchronous database interactions, and a custom brand/domain model to drive the News Studio UI and content adaptation pipelines.

Application Bootstrap and Configuration

The application entry point is `app/main.py`, where the `FastAPI` instance is initialized app/main.py:11. The bootstrap process involves registering middleware, mounting static assets for the UI, and including various API routers that handle news management, administrative tasks, and Instagram draft generation.

Configuration is managed via a centralized `Settings` class using Pydantic, which loads environment variables for database connections, API keys (e.g., Idealista), and operational limits app/config.py:10-45.

Key Components: * **Router Registration:** Routes are grouped into logical domains: `/api` for news app/main.py:25, `/api/admin` and `/api/tech/admin` for ingestion app/main.py:26-27, and `/api/ig` for social media drafts app/main.py:28. * **Lifecycle Hooks:** The application uses `@app.on_event("startup")` to initialize the database schema and `@app.on_event("shutdown")` to gracefully dispose of the database engine app/main.py:36-45.

For details, see Application Bootstrap and Configuration.

Sources: app/main.py:1-46, app/config.py:1-47

Middleware and Security

The service implements a multi-layered middleware stack to ensure performance and security: 1. **CORS:** Configured to allow cross-origin requests, facilitating the News Studio UI's interaction with the backend app/main.py:14-20. 2. **Rate Limiting:**

A custom `RateLimitMiddleware` differentiates between public and administrative traffic. Public endpoints are limited to 100 requests/minute, while admin endpoints are restricted to 10 requests/minute app/middleware.py:64-65, 98-101. 3. **Authentication:** Administrative and UI routes are protected. The UI utilizes a `basic_auth_dependency` that checks for `UI_USER` and `UI_PASS` environment variables app/auth.py:26-37.

Sources: app/middleware.py:1-131, app/main.py:13-23, app/auth.py:1-37

Database Layer

The persistence layer is built on PostgreSQL using SQLAlchemy's asynchronous extension. * **Engine & Session:** An `AsyncSessionLocal` factory provides scoped sessions for API requests app/database.py. * **Schema Management:** Alembic handles migrations, such as adding the `althara_content` JSONB field used for storing structured news data alembic/versions/43db6c86d1d4_add_althara_content_field.py:20-21. * **Data Validation:** Scripts like `check_structured_content.py` allow developers to audit the database state, verifying the presence of structured fields across the `News` table scripts/check_structured_content.py:19-32.

For details, see Database Layer.

Sources: app/database.py, alembic/versions/43db6c86d1d4_add_althara_content_field.py:1-26, scripts/check_structured_content.py:1-106

Brand and Domain Model

The system operates under a dual-brand strategy defined in `app/brands.py`. This configuration drives the logic for content filtering and UI themes.

Brand Key	Domain Name	Display Name	Target Industry
`althara`	`real_estate`	Althara	Real Estate / Market Trends
`oxono`	`tech`	Oxono	AI / Dev Tools / Security

The `BRANDS` registry app/brands.py:14-25 is the source of truth for the `get_domain_for_brand` and `get_brand_for_domain` utilities, which ensure that ingested news is correctly partitioned by its `domain` column in the database app/brands.py:28-51.

For details, see Brand and Domain Model.

Sources: app/brands.py:1-52, app/constants_tech.py:1-107

System Architecture Overview

The following diagram illustrates the flow from external sources through the FastAPI bootstrap and into the domain-specific logic.

Request and Ingestion Flow

```
graph TD
    subgraph "External Space"
        RSS["RSS Feeds (Wired, Genbeta, etc.)"]
        IDL["Idealista API"]
        Client["News Studio UI / Admin Scripts"]
    end

    subgraph "Code Entity Space: FastAPI Application"
        Main["app/main.py (FastAPI Instance)"]
        RLM["app/middleware.py (RateLimitMiddleware)"]
        Auth["app/auth.py (basic_auth_dependency)"]

        subgraph "Routers"
            NewsR["app/routers/news.py"]
            AdminR["app/routers/admin.py"]
            TechR["app/routers/tech_admin.py"]
        end
    end

    subgraph "Domain Logic"
        Brands["app/brands.py (BRANDS Registry)"]
        Config["app/config.py (Settings)"]
    end

    subgraph "Persistence"
        DB[("PostgreSQL (asyncpg)")]
        Models["app/models/news.py (News Model)"]
    end
```

```
    Client --> RLM
    RLM --> Auth
    Auth --> Main
    Main --> NewsR
    Main --> AdminR
    Main --> TechR

    AdminR --> Brands
    TechR --> Brands

    RSS --> AdminR
    IDL --> AdminR

    NewsR --> DB
    AdminR --> DB
    TechR --> DB
    DB --- Models
```

Sources: app/main.py:11-35, app/middleware.py:87-131, app/brands.py:14-25, app/config.py:10-45

Data Entity Mapping

```
graph LR
    subgraph "Natural Language Concepts"
        NewsItem["News Article"]
        IGPost["Instagram Post"]
        Brand["Brand (Althara/Oxono)"]
    end

    subgraph "Code Entities (SQLAlchemy / Pydantic)"
        NewsModel["app/models/news.py: News (ORM)"]
        IGModel["app/models/ig_draft.py: IGDraft (ORM)"]
        BrandDict["app/brands.py: BRANDS (Dict)"]
        NewsSchema["app/schemas/news.py: NewsRead (Pydantic)"]
    end

    NewsItem --- NewsModel
    IGPost --- IGModel
    Brand --- BrandDict
    NewsModel --- NewsSchema
    NewsModel -- "1:N Relationship" --> IGModel
```

Sources: app/models/news.py, app/brands.py:14-25, alembic/versions/43db6c86d1d4_add_althara_content_field.py:21

Application Bootstrap and Configuration

This section details the initialization and configuration lifecycle of the Althara News Service. It covers how the FastAPI application is instantiated, how settings are managed via Pydantic, the integration of specialized middleware for security and performance, and the lifecycle hooks that manage database connectivity.

Overview of Bootstrap Process

The application entry point is `app/main.py`, which orchestrates the assembly of the FastAPI instance, middleware stack, and routing table. The service follows a modular approach where specific domain logic (Real Estate vs. Tech) is isolated into separate routers but unified under a single application instance.

Application Lifecycle and Database Wiring

The service utilizes FastAPI's lifecycle events to manage the SQLAlchemy async engine. Upon startup, it attempts to synchronize the database schema (primarily for development/CI environments), and upon shutdown, it ensures all connections are gracefully terminated.

Title: Application Lifecycle Flow

```
graph TD
    subgraph "Startup Phase"
        A["main.py: startup()"] --> B["engine.begin()"]
        B --> C["Base.metadata.create_all"]
        C --> D["Database Ready"]
    end

    subgraph "Request Handling"
        D --> E["FastAPI App Instance"]
        E --> F["RateLimitMiddleware"]
        F --> G["CORSMiddleware"]
        G --> H["Router Dispatching"]
```

```
    end

    subgraph "Shutdown Phase"
        I["main.py: shutdown()"] --> J["engine.dispose()"]
        J --> K["Connections Closed"]
    end
```

Sources: app/main.py:11-46, app/database.py:6-6

Configuration Management

Configuration is handled by the `Settings` class in `app/config.py`, which leverages `pydantic-settings`. This model provides a type-safe way to load environment variables from a `.env` file or the system environment.

Key Configuration Parameters

Parameter	Type	Description
`DATABASE_URL`	`str`	Connection string for PostgreSQL (asyncpg).
`INGEST_TOKEN`	`Optional[str]`	Secret token required for admin ingestion endpoints.
`UI_USER` / `UI_PASS`	`Optional[str]`	Credentials for BasicAuth protection of the News Studio.
`TECH_RSS_LIMIT_PER_SOURCE`	`int`	Max items to fetch per tech source (Default: 5).
`REAL_ESTATE_RSS_LIMIT`	`int`	Max items to fetch per real estate source (Default: 10).
`AUTO_GENERATE_IG_AFTER_INGEST`	`bool`	If true, triggers the IG adapter immediately after news insertion.

Sources: app/config.py:10-45

Middleware Stack

The application implements a layered middleware stack to handle cross-cutting concerns before requests reach the route handlers.

1. CORS Middleware

Configured to allow broad access (`allow_origins=["*"]`) to facilitate frontend requests from various domains, though this is typically restricted in production environments. Sources: app/main.py:14-20

2. Rate Limiting Middleware

The `RateLimitMiddleware` (defined in `app/middleware.py`) implements an in-memory sliding window algorithm to protect the API from abuse. It distinguishes between public and administrative traffic.

- **Public Rate Limiter**: 100 requests per minute.
- **Admin Rate Limiter**: 10 requests per minute.

The middleware extracts the client IP via `get_client_ip` , checking `X-Forwarded-For` and `X-Real-IP` headers to support deployments behind proxies like Render or Nginx. Sources: app/middleware.py:62-131

Title: Middleware Data Flow

```
sequenceDiagram
    participant Client
    participant RLM as "RateLimitMiddleware"
    participant Router as "FastAPI Routers"

    Client->>RLM: HTTP Request
    Note over RLM: get_client_ip()
    alt Path starts with /api/admin
        RLM->>RLM: Use admin_rate_limiter (10 req/m)
    else Other paths
        RLM->>RLM: Use public_rate_limiter (100 req/m)
    end

    alt Limit Exceeded
        RLM-->>Client: 429 Too Many Requests
    else Allowed
```

```
        RLM->>Router: call_next(request)
        Router-->>RLM: Response
        RLM-->>Client: Response + X-RateLimit-Limit
    end
```

Sources: app/middleware.py:87-131, app/main.py:23-23

Router Registration and Static Assets

The application mounts several functional routers and a static file directory for the News Studio UI.

- **API Routers**:
  - `/api`: General news access via `news.router`.
  - `admin.router`: Real estate ingestion and maintenance.
  - `tech_admin.router`: Tech-specific ingestion (Oxono brand).
  - `ig_drafts.router`: Instagram draft management and publishing.
- **UI Router**: The `ui.router` serves the Jinja2 templates for the News Studio.
- **Static Files**: Mounted at `/static`, serving CSS and JS assets for the frontend.

Sources: app/main.py:25-34

Brand and Domain Mapping

The system supports a multi-brand architecture (Althara and Oxono) through `app/brands.py`. This configuration maps internal `domain` strings (`real_estate`, `tech`) to UI display names and CSS themes.

- **Althara**: Domain `real_estate`, Theme `althara`.
- **Oxono**: Domain `tech`, Theme `oxono`.

This mapping is critical for the `ui.py` router to render the correct brand context based on the URL path. Sources: app/brands.py:14-25

Database Layer

The Database Layer provides the asynchronous infrastructure required for persisting news entities and Instagram drafts. It leverages **SQLAlchemy 2.0** with the `asyncio` extension and uses `asyncpg` as the database driver to interface with PostgreSQL.

Core Infrastructure

The database configuration is centralized in `app/database.py`. It handles environment variable loading, URL normalization for asynchronous compatibility, and the creation of the global engine and session factory.

URL Normalization

Standard PostgreSQL connection strings (e.g., from managed services like Neon or Heroku) often use the `postgresql://` scheme and include parameters like `sslmode`. Because `asyncpg` requires the `postgresql+asyncpg://` dialect and manages SSL automatically, the system implements a normalization routine.

The `normalize_database_url` function performs the following transformations: 1. Replaces `postgresql://` with `postgresql+asyncpg://` app/database.py:23-24. 2. Parses and removes incompatible query parameters such as `sslmode` and `channel_binding` app/database.py:32-33. 3. Reconstructs the URL for use with `create_async_engine` app/database.py:40-47.

Engine and Session Factory

The `engine` is initialized with specific pooling configurations to optimize performance in a containerized environment: * **pool_pre_ping**: Enabled to verify connections before use app/database.py:72. * **pool_size**: Set to 5 connections app/database.py:73. * **max_overflow**: Allows up to 10 additional connections during bursts app/database.py:74.

The `AsyncSessionLocal` factory produces `AsyncSession` instances with `expire_on_commit=False` to prevent issues when accessing object attributes after a transaction is committed in an async context app/database.py:77-81.

The `get_db` Dependency

To manage session lifecycles within FastAPI routes, the `get_db` generator is used as a dependency. It ensures that a session is opened for every request and properly closed upon completion, even if exceptions occur app/database.py:87-94.

Sources: app/database.py:12-49, app/database.py:68-81, app/database.py:87-95

Data Flow: Connection Lifecycle

The following diagram illustrates how a request interacts with the database layer from normalization to session disposal.

Database Connection and Session Flow

```
sequenceDiagram
    participant Env as "Environment (OS/Dotenv)"
    participant DB as "app/database.py"
    participant Factory as "AsyncSessionLocal"
    participant Dep as "get_db (Dependency)"
    participant Route as "FastAPI Route"

    Env->>DB: "DATABASE_URL (raw)"
    DB->>DB: "normalize_database_url()"
    DB->>DB: "create_async_engine()"
    DB->>Factory: "Configures async_sessionmaker"

    Note over Dep, Route: Request Lifecycle
    Dep->>Factory: "Create new AsyncSession"
    Dep->>Route: "yield session"
    Route->>Route: "DB Operations (await)"
    Route-->>Dep: "Request Finished"
    Dep->>Dep: "await session.close()"
```

Sources: app/database.py:52-58, app/database.py:68-81, app/database.py:90-94

Declarative Models and Base

All ORM models in the service inherit from a single `Base` class, which is a `declarative_base` instance app/database.py:9. This allows SQLAlchemy to collect metadata for all tables, which is critical for Alembic migrations.

Entity Relationship Mapping

The system defines two primary entities: `News` and `IGDraft`. These are linked via a one-to-many relationship.

- **News**: The central entity representing ingested content app/models/news.py:9.
- **IGDraft**: Social media content derived from a `News` record app/models/ig_draft.py:9.

Entity Relationship Diagram

```
classDiagram
    class Base {
        <<DeclarativeBase>>
    }
    class News {
        "UUID id"
        "String title"
        "String domain"
        "JSON althara_content"
        "relationship ig_drafts"
    }
    class IGDraft {
        "UUID id"
        "UUID news_id (FK)"
        "JSONB carousel_slides"
        "String status"
        "relationship news"
    }

    Base <|-- News
    Base <|-- IGDraft
    News "1" -- "*" IGDraft : "cascade delete-orphan"
```

Sources: app/models/news.py:9-31, app/models/ig_draft.py:9-31, app/database.py:9

Migration Integration

The database layer is tightly integrated with Alembic for schema management. The `alembic/env.py` file mirrors the normalization logic found in `app/database.py` to ensure that migrations run correctly against the same asynchronous drivers.

Async Migration Execution

Alembic migrations are executed using the `run_async_migrations` function, which: 1. Retrieves the normalized URL via `get_url()` alembic/env.py:109. 2. Creates an engine using `async_engine_from_config` alembic/env.py:111. 3. Runs the synchronous migration functions within a `connection.run_sync` wrapper alembic/env.py:118.

Metadata Discovery

The `target_metadata` is imported directly from `app.database.Base.metadata` alembic/env.py:21. For Alembic to detect the models, the model files (`app/models/news.py` and `app/models/ig_draft.py`) are explicitly imported into the `env.py` script alembic/env.py:18-19.

Sources: alembic/env.py:24-61, alembic/env.py:101-121, alembic/env.py:17-21

Brand and Domain Model

The Althara News Service operates as a multi-tenant content engine supporting two distinct brands: **Althara** (Real Estate) and **Oxono** (Technology). This page describes the technical implementation of the brand registry, how logical domains map to physical database records, and how the API and UI layers use this model to provide partitioned experiences.

The Brand Registry

The system centralizes brand definitions in a single registry to ensure consistency across the ingestion pipeline, adaptation logic, and the News Studio UI.

Configuration Structure

Each brand is defined by a `BrandConfig` app/brands.py:8-12, which includes: - `domain` : The internal identifier used in the `news.domain` database column. - `display` : The human-readable name shown in the UI. - `theme` : A CSS/theme identifier used to switch visual styles in the frontend.

Brand Definitions

The `BRANDS` dictionary app/brands.py:14-25 serves as the source of truth:

Brand Key	Domain	Display Name	Theme	Primary Focus
`althara`	`real_estate`	Althara	althara	Real Estate market, laws, and trends.
`oxono`	`tech`	Oxono	oxono	AI, Dev tools, Security, and Infrastructure.

Helper Functions

The module provides utility functions to resolve mappings between brand keys and domains: - `get_domain_for_brand(brand)`: Maps a URL parameter (e.g., `oxono`) to a DB domain (e.g., `tech`) app/brands.py:28-31. - `get_brand_for_domain(domain)`: The inverse lookup used when processing news objects app/brands.py:46-51.

Sources: app/brands.py:1-52

Domain Filtering Flow

The `domain` field in the `News` model acts as the primary partition for all data. This allows a single database to store disparate content types while ensuring that an Althara user never sees Oxono technical updates.

Natural Language to Code Entity Space: Data Partitioning

The following diagram illustrates how the concept of "Brands" in the business domain maps to specific code structures and database fields.

```
graph TD
    subgraph "Natural Language Space"
        B1["Brand: Althara"]
        B2["Brand: Oxono"]
    end

    subgraph "Code Entity Space (app/brands.py)"
        REG["BRANDS Registry"]
        BC1["BrandConfig (domain='real_estate')"]
        BC2["BrandConfig (domain='tech')"]
    end

    subgraph "Database Space (app/models/news.py)"
        DB[("PostgreSQL")]
        COL["News.domain Column"]
    end

    B1 --> REG
    B2 --> REG
    REG --> BC1
    REG --> BC2
```

```
    BC1 --> COL
    BC2 --> COL
    COL --> DB
```

Sources: app/brands.py:14-25, app/models/news.py:16-100 (inferred from News model structure)

API and UI Integration

1. **UI Portal**: The `ui_portal` route app/routers/ui.py:43-44 takes a `{brand}` path parameter. It uses `get_domain_for_brand` app/routers/ui.py:59 to fetch the corresponding domain and filters the SQLAlchemy query app/routers/ui.py:66 so only relevant news is displayed.
2. **Category Mapping**: Depending on the domain, the UI dynamically switches the category labels displayed in filters. It uses `TECH_CATEGORY_LABELS` app/constants_tech.py:21-31 for the `tech` domain and `CATEGORY_LABELS` for `real_estate` app/routers/ui.py:97-101.
3. **Admin Operations**: Separate routers handle ingestion for different brands. For instance, `tech_admin.py` specifically targets the `tech` domain app/main.py:27.

Sources: app/routers/ui.py:43-118, app/main.py:25-28

Brand-Specific Logic

While the database schema is shared, the logic applied to each domain differs significantly based on the brand's target audience.

Oxono (Tech Domain)

The tech domain uses a specialized set of categories and guardrails defined in `constants_tech.py`. - **Categories**: Includes `AI_ML`, `SECURITY`, `TOOL_DISCOVERY`, etc. app/constants_tech.py:9-18. - **Guardrails**: Uses a strict "Allow List" approach. For a news item to be ingested, it must contain at least one `ALLOW_KEYWORDS` (e.g., "llm", "kubernetes", "vulnerability") app/constants_tech.py:89-103. - **Sources**: Ingests

from technical feeds like WIRED ES, Microsiervos, and developer newsletters app/constants_tech.py:41-75.

Althara (Real Estate Domain)

The real estate domain focuses on property market dynamics and regional news. - **Categories**: Uses the `AltharaCategoryV2` taxonomy (e.g., `MERCADO_RESIDENCIAL`, `LEGISLACION`) app/routers/ui.py:101. - **Data Enrichment**: Includes specific fields like `provincia` and `poblacion` which are often null for tech news but critical for real estate app/routers/ui.py:101.

Code Entity Space: UI Brand Routing

This diagram shows how the `ui.py` router handles the transition from the Brand Selector to brand-specific content views.

```
graph TD
    subgraph "app/routers/ui.py"
        START["GET /ui (ui_selector)"]
        PORTAL["GET /ui/{brand} (ui_portal)"]
        DETAIL["GET /ui/{brand}/news/{news_id} (ui_news_detail)"]

        FN1["get_domain_for_brand(brand)"]
        FN2["get_display_name(brand)"]
    end

    subgraph "Jinja2 Templates"
        T1["selector.html"]
        T2["portal.html"]
        T3["news_detail.html"]
    end

    START --> T1
    T1 -- "User clicks brand card" --> PORTAL
    PORTAL --> FN1
    PORTAL --> FN2
    PORTAL --> T2
    T2 -- "User clicks news item" --> DETAIL
    DETAIL --> T3
```

Sources: app/routers/ui.py:35-40, app/routers/ui.py:43-118, app/routers/ui.py:121-146, app/brands.py:28-38

Summary of Brand Assets

Asset	Althara (real_estate)	Oxono (tech)
Config File	`app/constants.py`	`app/constants_tech.py`
Admin Router	`app/routers/admin.py`	`app/routers/tech_admin.py`
UI Theme	`althara`	`oxono`
Primary Categories	Residential, Logistics, Macro	AI/ML, Security, DevTools

Sources: app/brands.py:14-25, app/main.py:26-27, app/routers/ui.py:97-101

News Ingestion Pipeline

The News Ingestion Pipeline is the entry point for all content in the Althara News Service. It is responsible for monitoring external sources, extracting relevant information, and transforming raw data into structured `News` entities. The pipeline supports two distinct brand domains: **Althara** (Real Estate) and **Oxono** (Tech).

Ingestion Architecture

The pipeline follows a multi-stage process: **Fetch → Extract → Filter → Classify → Score → Persist**. This ensures that only high-quality, relevant news reaches the database, while noise (like lifestyle or decoration articles) is discarded.

High-Level Data Flow

The following diagram illustrates the path from an external RSS feed or API to the `News` table in the database.

News Ingestion Data Flow

```
graph TD
    subgraph "External Sources"
        A1["RSS_SOURCES (Real Estate)"]
        A2["TECH_RSS_SOURCES (Tech)"]
        A3["Idealista API"]
    end

    subgraph "Ingestion Logic (app/ingestion/)"
        B1["rss_ingestor.py"]
        B2["tech_rss_ingestor.py"]
        B3["idealista_ingestor.py"]
    end

    subgraph "Processing & Validation"
        C1["passes_guardrails()"]
        C2["_extract_article_content()"]
        C3["_categorize_althara_v2()"]
        C4["_calculate_relevance_score()"]
    end
```

```
    subgraph "Persistence"
        D1[("Database: news table")]
    end

    A1 --> B1
    A2 --> B2
    A3 --> B3

    B1 & B2 & B3 --> C1
    C1 -- "Passed" --> C2
    C2 --> C3
    C3 --> C4
    C4 --> D1
```

Sources: app/ingestion/rss_ingestor.py:29-51, app/utils/guardrails.py:1-50

Real Estate Ingestion (Althara)

The Althara path focuses on the Spanish property market. It monitors professional economic press (Expansión, Cinco Días), real estate portals (Idealista, Fotocasa), and official government sources (BOE Subastas).

- **RSS Fetching:** Uses `feedparser` to read XML feeds and `httpx` for asynchronous requests.
- **Content Scraping:** Unlike simple RSS readers, the `_extract_article_content` function app/ingestion/rss_ingestor.py:56-122 uses `BeautifulSoup` to scrape the full article body, bypassing the limited snippets often found in RSS feeds.
- **Classification:** News is mapped to the `AltharaCategoryV2` taxonomy using keyword-based hints defined in `CATEGORY_HINTS`.

For details, see Real Estate RSS Ingestor.

Sources: app/ingestion/rss_ingestor.py:1-51, app/constants.py:22-59

Tech Ingestion (Oxono)

The Oxono path ingests technology and innovation news. It utilizes a separate set of sources and guardrails to maintain a distinct brand voice.

- **Sources:** Targeted feeds like WIRED España, Genbeta, and Xataka.
- **Tech Guardrails:** Uses `constants_tech.py` to filter for high-signal tech news, excluding general lifestyle or consumer electronics reviews that don't fit the professional profile.
- **Domain Tagging:** Every record is saved with `domain='tech'` to separate it from real estate content in the UI and API.

For details, see Tech RSS Ingestor (Oxono).

Idealista Ingestor

The `idealista_ingestor.py` provides a specialized path for property data. Since Idealista does not offer a public "news" API, this module interacts with their property search API via `idealista_client.py`.

- **Functionality:** It fetches specific property listings or market reports and converts them into `News` records.
- **Guardrails:** Applies standard real estate guardrails to ensure property descriptions meet the service's quality standards.

For details, see Idealista Ingestor.

Guardrails and Classification

The "Intelligence" of the pipeline resides in the `app/utils/guardrails.py` and `app/constants.py` modules. This layer acts as a gatekeeper.

- **Keyword Filtering:** The `passes_guardrails()` function app/utils/guardrails.py: 12-45 checks for `DENY_KEYWORDS` (e.g., "decoración", "interiorismo") and enforces `STRICT_REQUIRE_ALLOW` logic.

- **Relevance Scoring:** A `relevance_score` (0-100) is calculated based on category priority and the presence of high-value keywords like "precio", "récord", or "inversión" app/ingestion/rss_ingestor.py:142-160.
- **Taxonomy:** The system uses `AltharaCategoryV2`, a pro-level taxonomy designed to reduce ambiguity compared to the original V1 categories.

For details, see Content Guardrails and Classification.

Sources: app/constants.py:124-147, app/utils/guardrails.py:1-50

Code Entity Mapping

The following diagram bridges the logical ingestion steps to the specific classes and functions in the codebase.

Code Entity Mapping: Ingestion Pipeline

```
classDiagram
    class IngestorModule {
        <<file>>
        rss_ingestor.py
        tech_rss_ingestor.py
        idealista_ingestor.py
    }

    class LogicFunctions {
        _extract_article_content()
        _categorize_althara_v2()
        _calculate_relevance_score()
    }

    class Guardrails {
        passes_guardrails()
        DENY_KEYWORDS
        ALLOW_KEYWORDS
    }

    class DataModels {
        News (SQLAlchemy)
        AltharaCategoryV2 (Enum-like)
    }
```

```
    IngestorModule --> LogicFunctions : "Calls"
    LogicFunctions --> Guardrails : "Validates via"
    LogicFunctions --> DataModels : "Persists to"
```

Sources: app/ingestion/rss_ingestor.py:56-135, app/models/news.py:1-50, app/utils/guardrails.py:12-20

Real Estate RSS Ingestor

The Real Estate RSS Ingestor is the primary entry point for market news within the Althara domain. It is responsible for monitoring a curated list of industry sources, extracting high-fidelity content through web scraping, and applying specialized classification and scoring logic to ensure the feed remains focused on professional real estate investment and market trends.

Data Flow Overview

The ingestion process follows a linear pipeline from discovery to persistence: 1. **Discovery**: Iterates through `RSS_SOURCES` defined in app/ingestion/rss_ingestor.py:29-51. 2. **Fetch**: Uses `feedparser` to retrieve entry metadata and `httpx` for full-page scraping app/ingestion/rss_ingestor.py:67-71. 3. **Refinement**: Cleans HTML and extracts the core article body using `BeautifulSoup` app/ingestion/rss_ingestor.py:78-119. 4. **Validation**: Filters content against `DENY_KEYWORDS` and `ALLOW_KEYWORDS` app/ingestion/rss_ingestor.py:18-26. 5. **Classification**: Assigns an `AltharaCategoryV2` based on keyword density app/ingestion/rss_ingestor.py:125-135. 6. **Scoring**: Calculates a `relevance_score` (0-100) based on editorial priorities app/ingestion/rss_ingestor.py:142-160. 7. **Deduplication**: Checks for existing URLs in the `News` table before committing app/ingestion/rss_ingestor.py:13-15.

RSS Sources and Configuration

The system tracks a variety of sources ranging from economic press to official government bulletins. Each source is assigned a `default_category` to provide a fallback classification.

Source Type	Examples	Default Category
Economic Press	Expansion, Cinco Días	`SECTOR_INMOBILIARIO`
Real Estate Portals	Idealista, Fotocasa	`PRECIOS_VIVIENDA`
	Brainsre, EjePrime	`INVERSION_INSTITUCIONAL`

Source Type	Examples	Default Category
Professional Investment		
Official Bulletins	BOE Subastas	`BOE_SUBASTAS`
Appraisal Firms	Tinsa, Sociedad de Tasación	`PRECIOS_VIVIENDA`

Sources: app/ingestion/rss_ingestor.py:29-51

Content Extraction and Scraping

Unlike standard RSS readers that only ingest the provided snippet, this ingestor performs a "Deep Fetch" to retrieve the full article text. This is critical for subsequent NLP tasks and summary generation.

The `_extract_article_content` Function

This asynchronous function app/ingestion/rss_ingestor.py:56-123 performs the following: * **User-Agent Spoofing**: Mimics a modern browser to avoid blocks app/ingestion/rss_ingestor.py:69-70. * **Encoding Correction**: Attempts UTF-8 decoding with a Latin-1 fallback app/ingestion/rss_ingestor.py:74-77. * **Boilerplate Removal**: Decomposes `<script>`, `<style>`, `<nav>`, and `<footer>` tags app/ingestion/rss_ingestor.py:80-81. * **Heuristic Selection**: Searches for the article body using a prioritized list of CSS selectors (e.g., `.article-body`, `.post-content`) app/ingestion/rss_ingestor.py:85-100. * **Fallback Logic**: If no selectors match, it looks for `<div>` elements with high text density (>300 chars) app/ingestion/rss_ingestor.py:103-106.

Sources: app/ingestion/rss_ingestor.py:56-123, app/utils/html_utils.py:18-40

Logic and Classification Pipeline

The diagram below bridges the relationship between the raw data ingestion and the code entities responsible for refining it.

RSS Ingestion Logic Map

```
graph TD
    subgraph "External World"
```

```
        RSS["RSS Feeds (XML)"]
        Web["Article Webpages (HTML)"]
    end

    subgraph "app/ingestion/rss_ingestor.py"
        Fetch["_extract_article_content()"]
        Cat["_categorize_althara_v2()"]
        Score["_calculate_relevance()"]
    end

    subgraph "app/utils/"
        Guard["passes_guardrails()"]
        Clean["strip_html_tags()"]
    end

    subgraph "app/constants.py"
        Hints["CATEGORY_HINTS"]
        Priority["CATEGORY_PRIORITY_V2"]
    end

    RSS --> Fetch
    Web --> Fetch
    Fetch --> Clean
    Clean --> Guard
    Guard --> Cat
    Cat --> Hints
    Cat --> Score
    Score --> Priority
```

Sources: app/ingestion/rss_ingestor.py:56-160, app/utils/guardrails.py:10-31, app/constants.py:124-147

Guardrails and Filtering

The `passes_guardrails` utility app/utils/guardrails.py:10-31 ensures that irrelevant content (e.g., lifestyle, interior design, or ads) is discarded. * **Deny List**: If any keyword from `DENY_KEYWORDS` appears in the title or summary, the news is rejected app/utils/guardrails.py:25-27. * **Strict Mode**: When `STRICT_REQUIRE_ALLOW` is enabled, the news must contain at least one keyword from `ALLOW_KEYWORDS` to be persisted app/utils/guardrails.py:28-30.

Sources: app/utils/guardrails.py:10-31, app/ingestion/rss_ingestor.py:18-26

Classification and Relevance Scoring

AltharaCategoryV2 Classification

The function `_categorize_althara_v2` app/ingestion/rss_ingestor.py:125-135 performs a keyword frequency analysis against `CATEGORY_HINTS` app/constants.py:24. It selects the category with the highest match count, defaulting to `SECTOR_INMOBILIARIO` if no specific matches are found.

Relevance Scoring

Relevance is a numerical value (0-100) used to sort news in the UI. It is calculated by combining: 1. **Base Priority**: Derived from `CATEGORY_PRIORITY_V2` (e.g., `PRECIOS_VIVIENDA` = 90, `SECTOR_INMOBILIARIO` = 30) app/constants.py:124-147. 2. **Keyword Boost**: High-value terms like "récord", "sube", or "inversión" add weight app/ingestion/rss_ingestor.py:142-160.

Sources: app/ingestion/rss_ingestor.py:125-160, app/constants.py:61-87, app/constants.py:124-147

Persistence and Deduplication

Before a news item is saved as a `News` ORM entity, the ingestor checks for URL collisions to prevent duplicate entries in the database.

Entity Persistence Flow

```
graph LR
    subgraph "Database Entities"
        DB_News[("News Table")]
    end

    subgraph "Code Entities (app/models/news.py)"
        Model_News["class News"]
    end

    subgraph "Ingestor Action"
        Check["SQLAlchemy Select(url)"]
        Save["Session.add()"]
    end

    Check -- "Not Found" --> Save
```

```
    Save --> Model_News
    Model_News --> DB_News
```

Sources: app/ingestion/rss_ingestor.py:12-15, app/models/news.py:1-20

Tech RSS Ingestor (Oxono)

The **Tech RSS Ingestor** is the specialized ingestion engine for the **Oxono** brand. It is responsible for fetching, filtering, and classifying technology-related news to populate the platform's tech domain. Unlike the real estate ingestor, this component uses a distinct set of guardrails, a dedicated category taxonomy, and a specialized relevance scoring algorithm tailored for the tech industry.

Overview and Data Flow

The ingestor operates as an asynchronous pipeline that transforms raw RSS entries into structured `News` records with `domain='tech'`. The process is orchestrated by `ingest_tech_rss_sources` in app/ingestion/tech_rss_ingestor.py:25-121.

Tech Ingestion Pipeline

The following diagram illustrates the flow from external RSS feeds to the persistence layer:

Tech Ingestion Logic Flow

```
graph TD
    subgraph "External Sources"
        A["TECH_RSS_SOURCES"]
    end

    subgraph "app/ingestion/tech_rss_ingestor.py"
        B["ingest_tech_rss_sources()"]
        C["httpx.AsyncClient / feedparser"]
        D["passes_guardrails()"]
        E["URL Deduplication"]
        F["infer_category()"]
        G["compute_relevance_score()"]
        H["extract_tags()"]
    end

    subgraph "Database"
        I[("News Table (domain='tech')")]
```

```
    end

    A --> B
    B --> C
    C --> D
    D -- "Pass" --> E
    E -- "New URL" --> F
    F --> G
    G -- "Score >= MIN_SCORE" --> H
    H --> I
```

Sources: app/ingestion/tech_rss_ingestor.py:25-121, app/utils/guardrails.py:10-31, app/tech/classifier.py:11-137

Implementation Details

Configuration and Constants

The ingestor relies on `app.constants_tech` for its operational parameters: * **TECH_RSS_SOURCES**: A list of dictionaries containing the `name`, `url`, `source` label, and `default_category` for feeds like WIRED ES or Genbeta app/ingestion/tech_rss_ingestor.py:35-39. * **Guardrail Keywords**: `DENY_KEYWORDS`, `ALLOW_KEYWORDS`, and the `STRICT_REQUIRE_ALLOW` boolean flag used to filter out irrelevant content app/ingestion/tech_rss_ingestor.py:74-78. * **Thresholds**: `MIN_SCORE` (typically 50) defines the minimum relevance required for a news item to be saved app/ingestion/tech_rss_ingestor.py:93-94.

Classification and Scoring

The logic for processing tech content resides in `app/tech/classifier.py`.

Function	Responsibility	Logic Summary
`infer_category`	Assigns a `TechNewsCategory`	Keyword matching for AI/ML, Big Tech, Startups, Security, etc. app/tech/classifier.py:11-82
`extract_tags`	Generates comma-separated tags	Identifies tech terms (e.g., "llm", "saas") and company

Function	Responsibility	Logic Summary
		names app/tech/classifier.py:85-107
`compute_relevance_score`	Returns an integer (0-100)	Baseline of 50; boosts for AI/ML (+25) and specific keywords (+15) app/tech/classifier.py:110-136

Core Components Association

This diagram maps the functional steps of the ingestor to the specific code entities that implement them.

Code Entity Mapping

```
classDiagram
    class TechIngestor {
        +ingest_tech_rss_sources(session, max_items)
    }
    class Guardrails {
        +passes_guardrails(title, deny, allow, strict)
    }
    class TechClassifier {
        +infer_category(title, summary)
        +compute_relevance_score(title, summary, cat, source)
        +extract_tags(title, summary)
    }
    class NewsModel {
        +domain: "tech"
        +relevance_score: int
        +category: str
    }

    TechIngestor ..> Guardrails : "Uses [app/ingestion/tech_rss_ingestor.py:74]"
    TechIngestor ..> TechClassifier : "Uses [app/ingestion/tech_rss_ingestor.py:87-89]"
    TechIngestor ..> NewsModel : "Persists [app/ingestion/tech_rss_ingestor.py:98-110]"
```

Sources: app/ingestion/tech_rss_ingestor.py:1-121, app/tech/classifier.py:1-137, app/models/news.py:1-110

Execution Logic

1. **Fetching**: The ingestor uses `httpx` to fetch the XML feed and `feedparser` to parse the structure app/ingestion/tech_rss_ingestor.py:44-52.
2. **Cleaning**: HTML tags are stripped from the summary or description using `strip_html_tags` app/ingestion/tech_rss_ingestor.py:69-72.
3. **Filtering**:
    - **Guardrails**: Checks if the title/summary contains forbidden keywords or meets strict requirements app/ingestion/tech_rss_ingestor.py:74-78.
    - **Deduplication**: Queries the database by URL to ensure the item hasn't been processed previously app/ingestion/tech_rss_ingestor.py:81-84.
4. **Enrichment**:
    - `published_at` is normalized using `parse_published_date` app/ingestion/tech_rss_ingestor.py:86.
    - `category` and `relevance_score` are computed based on tech-specific logic app/ingestion/tech_rss_ingestor.py:87-91.
5. **Persistence**: Items meeting the `MIN_SCORE` are instantiated as `News` objects with `domain="tech"` and committed to the database app/ingestion/tech_rss_ingestor.py:98-119.

Sources: app/ingestion/tech_rss_ingestor.py:25-121, app/tech/classifier.py:110-136, app/utils/guardrails.py:10-31

Idealista Ingestor

The Idealista Ingestor is a specialized component of the News Service designed to fetch real estate market data and news from Idealista. It consists of a client for handling API interactions and an ingestor service that processes the data, applies guardrails, and persists it to the database.

Overview and Limitations

It is important to note that the official Idealista API primarily focuses on property search and market data rather than a public news endpoint app/ingestion/idealista_client.py:4-6. Consequently, the current implementation serves as a structured foundation for future integration, utilizing a mock mechanism for development and testing while maintaining the data structures required for real estate news app/ingestion/idealista_client.py:7-12.

Core Components

Component	File	Responsibility
`IdealistaClient`	app/ingestion/idealista_client.py:32-33	Manages API credentials and data retrieval logic.
`IdealistaNewsItem`	app/ingestion/idealista_client.py:21-22	Pydantic model defining the schema for ingested Idealista data.
`ingest_idealista_news`	app/ingestion/idealista_ingestor.py:15-15	Orchestrates fetching, filtering via guardrails, and DB insertion.

Sources: app/ingestion/idealista_client.py:4-33, app/ingestion/idealista_ingestor.py:15-15

Data Flow and Logic

The ingestion process follows a strict pipeline to ensure that only relevant, high-quality real estate news enters the system.

Ingestion Pipeline Diagram

The following diagram illustrates the flow from the `IdealistaClient` through the guardrails check to the `News` ORM model.

"Idealista Ingestion Pipeline"

```
graph TD
    subgraph "Code Entity Space"
        A["IdealistaClient.fetch_news()"] --> B["IdealistaNewsItem (List)"]
        B --> C["passes_guardrails()"]
        C -- "True" --> D["select(News).where(url)"]
        D -- "Not Found" --> E["News (SQLAlchemy Model)"]
        E --> F["session.add() & session.commit()"]
        C -- "False" --> G["Discard Item"]
    end

    subgraph "External/Data Space"
        H[("Idealista API / Mock")] -.-> A
        F --> I[("Database Table: news")]
    end
```

Sources: app/ingestion/idealista_ingestor.py:15-61, app/ingestion/idealista_client.py:41-53

Implementation Details

1. **Fetching**: The `ingest_idealista_news` function initializes an `IdealistaClient` and calls `fetch_news(limit=20)` app/ingestion/idealista_ingestor.py:26-27.
2. **Guardrails**: Each item is passed through `passes_guardrails`. This utility checks the title, summary, and URL against `DENY_KEYWORDS`, `ALLOW_KEYWORDS`, and the `STRICT_REQUIRE_ALLOW` flag app/ingestion/idealista_ingestor.py:33-37.
3. **Deduplication**: The ingestor performs a uniqueness check by querying the `News` table for the specific URL using `select(News).where(News.url == item.url)` app/ingestion/idealista_ingestor.py:39-41.

4. **Persistence**: If the news item is new and passes filters, it is mapped to a `News` ORM object and saved with `domain` (implied real estate) and specific fields like `provincia` or `poblacion` if available app/ingestion/idealista_ingestor.py:43-56.

Sources: app/ingestion/idealista_ingestor.py:26-61, app/utils/guardrails.py:1-11

Technical Reference

IdealistaClient Configuration

The client retrieves its configuration from the global `settings` object, which maps to environment variables: * `IDEALISTA_API_BASE_URL` app/ingestion/idealista_client.py:37 * `IDEALISTA_API_KEY` app/ingestion/idealista_client.py:38 * `IDEALISTA_API_SECRET` app/ingestion/idealista_client.py:39

News Item Schema

The `IdealistaNewsItem` Pydantic model ensures data consistency before the ingestor attempts to transform it into a database record.

Field	Type	Description
`title`	`str`	Headline of the news item app/ingestion/idealista_client.py:23
`source`	`str`	Hardcoded to "Idealista" app/ingestion/idealista_client.py:24
`url`	`str`	Unique identifier and link app/ingestion/idealista_client.py:25
`published_at`	`datetime`	Publication timestamp app/ingestion/idealista_client.py:26
`category`	`str`	Mapped to `NewsCategory` constants app/ingestion/idealista_client.py:27
`raw_summary`	`Optional[str]`	

Field	Type	Description
		Brief excerpt of the content app/ingestion/idealista_client.py:28

Code Mapping: Client to Ingestor

"Entity Mapping and Flow"

```
classDiagram
    class IdealistaClient {
        +fetch_news(limit)
    }
    class IdealistaNewsItem {
        +title: str
        +url: str
        +category: str
    }
    class ingest_idealista_news {
        <<function>>
        +session: AsyncSession
    }
    class News {
        <<ORM>>
        +id: int
        +title: str
        +url: str
    }

    IdealistaClient ..> IdealistaNewsItem : produces
    ingest_idealista_news --> IdealistaClient : uses
    ingest_idealista_news --> News : creates
```

Sources: app/ingestion/idealista_client.py:21-41, app/ingestion/idealista_ingestor.py:15-56, app/models/news.py:1-20

Content Guardrails and Classification

This section details the logic used to filter, categorize, and score ingested news content. The system employs a multi-layered approach using keyword-based guardrails to prevent "noise" (e.g., lifestyle or decor content) and a heuristic classification engine to map articles to the **AltharaCategoryV2** taxonomy.

Content Guardrails

The guardrail system ensures that only relevant news enters the pipeline. This is primarily handled by the `passes_guardrails()` utility function.

Guardrail Logic

The function `passes_guardrails()` evaluates the `title`, `summary`, and `url` of an article against predefined lists of allowed and denied keywords app/utils/guardrails.py:10-17.

1. **Normalization**: All text and keywords are converted to lowercase for case-insensitive matching app/utils/guardrails.py:24.
2. **Deny List**: If any keyword from the `deny_keywords` list is found, the article is immediately rejected app/utils/guardrails.py:25-27.
3. **Strict Requirement**: If `strict_require_allow` is enabled, the article must contain at least one keyword from the `allow_keywords` list to pass app/utils/guardrails.py:28-30.

Keyword Configuration

For the Althara (Real Estate) brand, these lists are defined in `app/constants.py`: * **DENY_KEYWORDS**: Includes terms related to interior design, lifestyle, and non-professional content (e.g., "decoración", "sofá", "plantas", "limpiar") app/constants.py:180-200. * **ALLOW_KEYWORDS**: Focuses on professional market indicators (e.g., "vivienda", "hipoteca", "alquiler", "euribor", "cnmv") app/constants.py:202-220. * **STRICT_REQUIRE_ALLOW**: Set to `True` for Althara to ensure high signal-to-noise ratio app/constants.py:222.

Data Flow: Guardrail Validation

The following diagram illustrates how the `rss_ingestor.py` uses the guardrails during the ingestion loop.

Guardrail Execution Flow

```
graph TD
    subgraph "rss_ingestor.py"
        A["fetch_rss_news()"] --> B["Loop: Entry in Feed"]
        B --> C["passes_guardrails()"]
    end

    subgraph "guardrails.py"
        C --> D{"Contains DENY_KEYWORDS?"}
        D -- "Yes" --> E["Return False (Discard)"]
        D -- "No" --> F{"STRICT_REQUIRE_ALLOW?"}
        F -- "Yes" --> G{"Contains ALLOW_KEYWORDS?"}
        G -- "No" --> E
        G -- "Yes" --> H["Return True (Proceed)"]
        F -- "No" --> H
    end

    H --> I["_extract_article_content()"]
    I --> J["_categorize_althara_v2()"]
```

Sources: app/utils/guardrails.py:10-31, app/ingestion/rss_ingestor.py:18-26, app/constants.py:180-222

Classification Taxonomy (AltharaCategoryV2)

The service uses a "Pro" taxonomy for real estate, designed to minimize ambiguity and focus on macro-economic topics.

Category Groups

The `AltharaCategoryV2` class defines the stable macro-topics app/constants.py:22-59:

Group	Categories
Market	`MERCADO_COMPRAVENTA`, `PRECIOS_VIVIENDA`, `ALQUILER_RESIDENCIAL`, `OFERTA_Y_STOCK`
Financing	`HIPOTECAS_Y_CREDITO`, `TIPOS_Y_MACRO`
Investment	`INVERSION_INSTITUCIONAL`, `OPERACIONES_CORPORATIVAS`, `GRANDES_TENEDORES`
Regulation	`REGULACION_VIVIENDA`, `URBANISMO_Y_PLANEAMIENTO`, `BOE_SUBASTAS`
Construction	`CONSTRUCCION_Y_COSTES`, `INDUSTRIALIZACION_MODULAR`

Heuristic Classification

Classification is performed by `_categorize_althara_v2()` in `rss_ingestor.py`. It uses `CATEGORY_HINTS` (a mapping of categories to specific keyword lists) to score the article app/ingestion/rss_ingestor.py:125-135.

1. The function combines the title and summary into a single text block.
2. It iterates through `CATEGORY_HINTS`.
3. The category with the highest number of keyword matches is selected.
4. If no hints match, it defaults to `SECTOR_INMOBILIARIO` app/ingestion/rss_ingestor.py:128.

Sources: app/constants.py:22-59, app/ingestion/rss_ingestor.py:125-135

Relevance Scoring

To prioritize news in the UI and for social media selection, the system computes a `relevance_score` (0-100).

Scoring Components

The score is calculated in `_calculate_relevance_score()` using two main factors app/ingestion/rss_ingestor.py:155-175:

1. **Category Priority**: Each category has a base weight defined in `CATEGORY_PRIORITY_V2`. For example, `PRECIOS_VIVIENDA` (90) is ranked higher than `BOE_SUBASTAS` (55) app/constants.py:124-147.
2. **Keyword Boost**: Specific high-value keywords (e.g., "récord", "cae", "burbuja") add a fixed bonus (typically +10) to the score, capped at 100 app/ingestion/rss_ingestor.py:165-173.

Scoring Entity Map

```
classDiagram
    class AltharaCategoryV2 {
        <<enumeration>>
        PRECIOS_VIVIENDA
        HIPOTECAS_Y_CREDITO
        INVERSION_INSTITUCIONAL
    }
    class CATEGORY_PRIORITY_V2 {
        <<dict>>
        PRECIOS_VIVIENDA: 90
        SECTOR_INMOBILIARIO: 30
    }
    class ScoringLogic {
        <<function>>
        _calculate_relevance_score()
    }

    ScoringLogic ..> CATEGORY_PRIORITY_V2 : "reads base weight"
    ScoringLogic ..> AltharaCategoryV2 : "identifies category"
    ScoringLogic ..> HIGH_VALUE_KEYWORDS : "checks for +10 bonus"
```

Sources: app/constants.py:124-147, app/ingestion/rss_ingestor.py:138-175

Migration: V1 to V2

The system maintains backward compatibility and supports legacy data through the `V1_TO_V2_MAP`.

Migration Logic

When processing older records or integrating with systems that still use the V1 taxonomy, the `V1_TO_V2_MAP` translates legacy strings to the new `AltharaCategoryV2` values app/constants.py:157-175.

Mapping Examples: * `FONDOS_INVERSION_INMOBILIARIA` → `INVERSION_INSTITUCIONAL` app/constants.py:159 * `NOTICIAS_HIPOTECAS` → `HIPOTECAS_Y_CREDITO` app/constants.py:171 * `BURBUJA_INMOBILIARIA` → `PRECIOS_VIVIENDA` app/constants.py:168

This map is used by scripts like `recategorize_news.py` to update the database schema without losing historical context.

Sources: app/constants.py:151-175

Content Adaptation and Transformation

The **Content Adaptation and Transformation** layer is responsible for converting raw, ingested news data into high-value, brand-aligned content. This process moves beyond simple data storage, applying sophisticated text processing and strategic narrative reconstruction to ensure all output matches the professional and analytical voice of the **Althara** (Real Estate) or **Oxono** (Tech) brands.

This layer acts as the bridge between the **Ingestion Pipeline** (which fetches raw data) and the **API/UI** (which serves the final content to users and social media managers).

High-Level Transformation Flow

The transformation process follows a structured pipeline that cleans raw input, extracts key entities, and generates multi-format outputs including professional summaries, structured JSON for web components, and multi-slide Instagram drafts.

```
graph TD
    subgraph "Natural Language Space"
        A["Raw RSS Content (HTML)"] --> B["Cleaned Plain Text"]
        B --> C["Strategic Narrative"]
    end

    subgraph "Code Entity Space"
        B -- "_clean_html()" --> D["news_adapter.py"]
        C -- "build_althara_summary()" --> E["News.althara_summary"]
        C -- "build_all_content_structured()" --> F["News.althara_content (JSONB)"]
        F -- "generate_draft()" --> G["ig_adapter.py"]
        G --> H["IGDraft Record"]
    end

    D --> E
    D --> F
```

Sources: app/adapters/news_adapter.py:24-95, app/adapters/news_adapter.py:214-233, app/adapters/news_adapter.py:330-363

Core Components

1. Althara News Adapter

The primary engine for the Real Estate domain. It transforms raw summaries into a three-tier narrative: a cold factual description, a strategic category-based reading, and a brand-specific "closer." It also produces a complex JSON structure (`althara_content`) used for advanced UI rendering.

- **Key Functions**: `build_althara_summary`, `build_all_content_structured`.
- **Key Logic**: Mapping news categories (e.g., `PRECIOS_VIVIENDA`) to specific strategic insights.
- **Details**: For a deep dive into HTML cleaning and the v2.0 JSON schema, see Althara News Adapter.

2. Instagram Draft Generator

A specialized adapter that takes adapted news content and composes it into social-media-ready formats. It handles the creation of multi-slide carousels (Hecho/Contexto/Cierre) and generates optimized captions with hashtags.

- **Key Functions**: `generate_variants`, `compose_instagram_post`.
- **Key Logic**: Deterministic seed-based generation to ensure consistent variants for the same news item.
- **Details**: For details on slide composition and brand-specific logic, see Instagram Draft Generator.

3. Text Utilities

A collection of low-level modules providing the "mechanical" text processing required by the adapters. This includes regex-based HTML stripping, sentence-level text compaction, and date parsing.

- **Modules**: `html_utils.py`, `text_compaction.py`, `rss_utils.py`.
- **Details**: For utility function signatures and logic, see Text Utilities.

Data Evolution Table

The following table illustrates how a single news item evolves through the adaptation layer:

Stage	Field/Entity	Description
Ingested	`News.raw_summary`	Raw HTML content directly from the RSS feed.
Adapted	`News.althara_summary`	Plain text summary following the Fact -> Strategy -> Closer pattern.
Structured	`News.althara_content`	JSONB field containing `hecho`, `lectura`, `implicaciones`, and `senales_a_vigilar`.
Social	`IGDraft`	A separate table record containing `carousel_slides`, `caption`, and `hashtags`.

Sources: app/adapters/news_adapter.py:214-233, app/adapters/news_adapter.py:330-363

Adaptation Logic Mapping

The system maps internal code identifiers to specific narrative strategies used during transformation.

```
graph LR
    subgraph "Code Entity Space"
        CAT1["PRECIOS_VIVIENDA"]
        CAT2["INVERSION_INSTITUCIONAL"]
        CAT3["NORMATIVAS_VIVIENDAS"]
    end

    subgraph "Narrative Strategy"
        CAT1 --> S1["Supply/Demand Risk Adjustment"]
        CAT2 --> S2["Silent Capital Rotation"]
        CAT3 --> S3["Market Access Reordering"]
    end

    S1 -- "_build_strategic_line()" --> OUT["Althara Summary"]
```

```
    S2 --> OUT
    S3 --> OUT
```

Sources: app/adapters/news_adapter.py:121-178

Child Pages

- Althara News Adapter — Deep dive into `news_adapter.py`: entrypoints, HTML cleaning, and strategic line mapping.
- Instagram Draft Generator — How `ig_adapter.py` generates `IGDraft` records and carousel slides.
- Text Utilities — Supporting modules for text compaction, HTML stripping, and date normalization.

Althara News Adapter

The **Althara News Adapter** is the core transformation engine responsible for converting raw, scraped news data into a structured, brand-aligned format. It specializes in the "Althara" brand voice—characterized by an analytical, professional, and strategic tone—and prepares content for both the web portal and automated social media distribution.

Purpose and Scope

The adapter acts as a bridge between the raw ingestion layer and the presentation layer. It performs three primary functions: 1. **Normalization**: Cleaning noisy HTML and metadata from RSS feeds. 2. **Narrative Reconstruction**: Transforming headlines into a three-part narrative (Fact, Strategy, Closer). 3. **Structured Extraction**: Identifying key data points (facts, implications, signals) to populate the `althara_content` JSONB field, supporting the v2.0 schema used by the UI and the Instagram generator.

Implementation Details

The logic is centralized in `app/adapters/news_adapter.py`. It follows a deterministic pipeline to ensure consistency across different news items within the same category.

Data Flow: Raw to Structured

The transformation process follows a linear path from raw input to a rich JSON object.

```
graph TD
    subgraph "Input Space"
        A["News Record (Raw)"] --> B["raw_summary"]
        A --> C["title"]
        A --> D["category"]
    end
```

```
    subgraph "news_adapter.py"
        B --> E["_clean_html()"]
        E --> F["build_structured_content()"]
        C --> F
        D --> F

        F --> G["_build_fact_line()"]
        F --> H["_build_strategic_line()"]
        F --> I["_pick_closer()"]
        F --> J["_extract_key_data()"]
    end

    subgraph "Output Space (v2.0 Schema)"
        G --> K["hecho"]
        H --> L["lectura"]
        J --> M["implicaciones"]
        J --> N["senales_a_vigilar"]
        I --> O["althara_summary (Legacy)"]
    end

    K & L & M & N --> P["News.althara_content (JSONB)"]
```

Sources: app/adapters/news_adapter.py:7-21, app/adapters/news_adapter.py:215-263

HTML Cleaning and Normalization

Raw RSS content often contains tracking pixels, social sharing prompts, and encoding artifacts (mojibake). The `_clean_html` function applies a multi-pass cleaning strategy: * **Tag Stripping**: Uses `strip_html_tags` from `html_utils.py` app/adapters/news_adapter.py:29. * **Metadata Removal**: Regex patterns target common newsroom signatures like "Compartir en Facebook" or "DREAMSTIME" app/adapters/news_adapter.py:33-46. * **Orthographic Fixes**: Hardcoded mappings fix common Spanish accentuation errors found in automated scraping (e.g., "psicologa" → "psicóloga") app/adapters/news_adapter.py:59-90.

Sources: app/adapters/news_adapter.py:24-95, app/utils/html_utils.py:18-40

Core Entrypoints

The adapter provides several high-level functions used by the `admin` and `ig_drafts` routers.

Function	Role	Output
`build_all_content_structured`	Main pipeline entrypoint	Updates `althara_summary`, `instagram_post`, and `althara_content`
`build_structured_content`	Schema v2.0 generator	Returns a Dict with `hecho`, `lectura`, `implicaciones`, `senales_a_vigilar`
`build_althara_summary`	Narrative generator	A single string combining Fact + Strategy + Closer
`build_instagram_post`	Legacy IG formatter	A pre-formatted caption for social media

Sources: app/adapters/news_adapter.py:215-263, app/adapters/news_adapter.py:266-288

Narrative Construction Logic

Althara's voice is built using a "Strategic Line Mapping" technique. The system interprets the `category` of the news to provide context that goes beyond the headline.

1. The Fact Line (`hecho`)

Builds a cold, objective description. If the text doesn't start with an article (El, La, En), it prepends a formal introduction: "Los últimos datos apuntan a lo siguiente..." app/adapters/news_adapter.py:98-118.

2. The Strategic Line (`lectura`)

The adapter uses a mapping of `AltharaCategoryV2` values to specific analytical insights. * **PRECIOS_VIVIENDA**: Focuses on the gap between supply and demand app/adapters/news_adapter.py:130-133. * **FONDOS_INVERSION**: Interprets moves as "silent rotations of capital" app/adapters/news_adapter.py:135-138. * **HIPOTECAS_Y_CREDITO**: Discusses the redefinition of credit advantage app/adapters/news_adapter.py:140-143.

3. The Closer

A selection from `ALTHARA_CLOSERS` is chosen using a deterministic seed (usually the news ID) to ensure that the same news item always generates the same summary unless the seed changes app/adapters/news_adapter.py:180-192.

Sources: app/adapters/news_adapter.py:16-21, app/adapters/news_adapter.py:121-177

JSON Schema v2.0 (althara_content)

The modern UI and the `ig_adapter.py` rely on the `althara_content` JSONB field. This field decomposes the news into actionable intelligence:

```
{
  "hecho": "Descripción técnica y fría del suceso...",
  "lectura": "Análisis estratégico de por qué esto importa...",
  "implicaciones": [
    "Punto de impacto 1",
    "Punto de impacto 2"
  ],
  "senales_a_vigilar": [
    "Métrica o evento futuro a observar"
  ]
}
```

The extraction of `implicaciones` and `senales_a_vigilar` utilizes `extract_key_sentences` from `text_compaction.py`, which prioritizes sentences

containing numerical data or specific market keywords app/adapters/news_adapter.py:195-212.

Sources: app/adapters/news_adapter.py:236-241, app/utils/text_compaction.py:47-85

System Mapping: Code to Concept

This diagram maps the logical transformation steps to the specific Python functions and utilities that execute them.

```
graph LR
    subgraph "Data Transformation Layer"
        direction TB
        F1["_clean_html"] -- "Uses" --> U1["strip_html_tags"]
        F2["build_structured_content"] -- "Calls" --> F3["_build_fact_line"]
        F2 -- "Calls" --> F4["_build_strategic_line"]
        F2 -- "Calls" --> F5["_extract_key_data"]
        F5 -- "Uses" --> U2["extract_key_sentences"]
    end

    subgraph "Code Entities (app/)"
        U1["utils/html_utils.py"]
        U2["utils/text_compaction.py"]
        F1 & F2 & F3 & F4 & F5["adapters/news_adapter.py"]
    end

    subgraph "Logical Output"
        O1["Clean Narrative"]
        O2["Strategic Reading"]
        O3["Actionable Signals"]
    end

    F3 --> O1
    F4 --> O2
    F5 --> O3
```

Sources: app/adapters/news_adapter.py:24-95, app/adapters/news_adapter.py:231-255, app/utils/text_compaction.py:47-52

QA and Validation

The adapter includes basic guardrails during content generation: * **Length Constraints**: Fact lines are truncated at 220 characters using `shorten` app/adapters/news_adapter.py:107. * **Sentence Integrity**: Uses `truncate_at_sentence` to ensure that summaries never end mid-word or mid-thought app/utils/text_compaction.py:20-44. * **Empty States**: If a `raw_summary` is missing, the adapter falls back to using the `title` as the base for all structured fields app/adapters/news_adapter.py:108-109.

Sources: app/adapters/news_adapter.py:100-110, app/utils/text_compaction.py:20-44

Instagram Draft Generator

The Instagram Draft Generator is a specialized adaptation pipeline implemented in `ig_adapter.py` that transforms ingested news into structured social media content. It utilizes an **Extract + Compose** strategy to ensure that carousel slides and captions maintain high-quality, brand-aligned messaging without mid-sentence truncation app/adapters/ig_adapter.py:1-7.

Extract + Compose Pipeline

The generator operates by first extracting semantic facts from raw content and then composing them into brand-specific formats. Unlike simple summarization, this pipeline prioritizes complete sentences and "quiet luxury" (Althara) or "systems thinking" (Oxono) tones app/adapters/ig_adapter.py:5-7.

Data Flow Diagram

The following diagram illustrates how raw news data is processed into an `IGDraft` entity.

IG Draft Generation Flow

```
graph TD
    subgraph "Natural Language Space"
        A["Raw News (Title + Summary)"]
        B["Brand Voice (Althara/Oxono)"]
    end

    subgraph "Code Entity Space (app/adapters/ig_adapter.py)"
        C["generate_ig_draft()"]
        D["_extract_althara() / _extract_oxono()"]
        E["_compose_slides_althara() / _compose_slides_oxono()"]
        F["_compose_caption_althara() / _compose_caption_oxono()"]
    end

    subgraph "Database Entity Space"
        G["IGDraft Record (status=DRAFT)"]
    end
```

```
    A --> C
    B --> C
    C --> D
    D --> E
    D --> F
    E --> G
    F --> G
```

Sources: app/adapters/ig_adapter.py:218-245, app/adapters/ig_adapter.py:53-73, app/adapters/ig_adapter.py:149-156

Carousel Slide Composition

The system generates exactly **3 slides** for every carousel, following a specific narrative arc app/adapters/ig_adapter.py:32-32.

1. **Slide 1: Hecho (Fact)** - The primary news event.
2. **Slide 2: Contexto (Context)** - Supporting details or implications.
3. **Slide 3: Cierre (Closing)** - A strategic takeaway or Call to Action (CTA).

Each slide body is strictly limited to **110 characters** to ensure readability on mobile devices app/adapters/ig_adapter.py:31-31. The function `_to_slide_body` uses `truncate_at_sentence` from text utilities to prevent cutting words in half app/adapters/ig_adapter.py:44-49.

Brand	Slide 1 Title	Slide 2 Title	Slide 3 Title
Althara	Hecho	Contexto	Cierre
Oxono	Tesis	Hecho	Cierre

Sources: app/adapters/ig_adapter.py:81-103, app/adapters/ig_adapter.py:159-176, app/utils/text_compaction.py:20-44

Brand-Specific Logic

The generator applies distinct logic based on the target brand, defined by the `domain` of the news item app/adapters/ig_adapter.py:228-232.

Althara (Real Estate)

- **Tone**: Professional, analytical, "Quiet Luxury".
- **Extraction**: Uses `_extract_althara` to pull sentences that fit the 110-character limit app/adapters/ig_adapter.py:53-60.
- **Strategic Reading**: Incorporates a "lectura" (strategic line) based on the news category app/adapters/ig_adapter.py:62-73.
- **Caption**: Maximum 900 characters, including a hook, bullet points, and a source line app/adapters/ig_adapter.py:34-34, app/adapters/ig_adapter.py:106-122.

Oxono (Tech)

- **Tone**: Systems thinking, operational, direct.
- **Extraction**: Focuses on technical impact and execution app/adapters/ig_adapter.py:149-156.
- **Caption**: Range of 500-850 characters with technical takeaways like "Validar supuestos" app/adapters/ig_adapter.py:38-39, app/adapters/ig_adapter.py:179-193.

Sources: app/adapters/ig_adapter.py:53-144, app/adapters/ig_adapter.py:149-213

Hashtag Generation and Determinism

Hashtags are generated dynamically based on the news category and a provided `seed` app/adapters/ig_adapter.py:125-144.

- **Seed-based Determinism**: The `seed` (often derived from the News ID or timestamp) ensures that the same news item produces consistent hashtags and CTAs across multiple generation calls app/adapters/ig_adapter.py:180-183.
- **Category Mapping**: Specific categories like `PRECIOS_VIVIENDA` or `AI_ML` trigger relevant tags (e.g., `#preciosvivienda`, `#ia`) app/adapters/ig_adapter.py:127-133, app/adapters/ig_adapter.py:198-202.
- **Quantity**: Althara targets 8-12 hashtags, while Oxono targets 6-10 app/adapters/ig_adapter.py:35-41.

Sources: app/adapters/ig_adapter.py:125-144, app/adapters/ig_adapter.py:196-213

Key Functions and Entities

`generate_ig_draft()`

The main entry point for the adapter. It takes a `News` object and returns a dictionary structured for the `IGDraft` model app/adapters/ig_adapter.py:218-245.

`generate_variants()`

This function creates multiple versions of a draft for the same news item by varying the `seed`. This allows editors to choose between different hooks or CTA combinations app/adapters/ig_adapter.py:248-258.

`IGDraft` Model

The resulting record is stored in the `ig_drafts` table with the following structure: * `news_id`: Foreign key to the parent news item alembic/versions/002_add_domain_relevance_ig_drafts.py:29-29. * `carousel_slides`: JSONB field containing the title and body for the 3 slides alembic/versions/002_add_domain_relevance_ig_drafts.py:31-31. * `status`: Lifecycle state (e.g., `DRAFT`, `NEEDS_REVIEW`, `APPROVED`) alembic/versions/002_add_domain_relevance_ig_drafts.py:39-39.

Entity Mapping: IG Adapter to Database

```
classDiagram
    class ig_adapter {
        +generate_ig_draft(news, brand)
        +generate_variants(news, brand, count)
        -_compose_slides_althara()
        -_compose_caption_althara()
    }
    class IGDraft {
        +UUID id
        +UUID news_id
        +JSONB carousel_slides
        +Text caption
        +JSONB hashtags
        +Text status
    }
    ig_adapter ..> IGDraft : creates
```

Sources: app/adapters/ig_adapter.py:218-258, alembic/versions/002_add_domain_relevance_ig_drafts.py:26-46

Text Utilities

Text utilities provide the foundational logic for cleaning, parsing, and compacting content across the news service. These modules ensure that raw data from RSS feeds is sanitized for the database and subsequently transformed into concise, brand-aligned formats for social media and web display.

HTML and Text Cleaning

The system uses `app/utils/html_utils.py` to sanitize incoming content. This is critical for both the ingestion pipeline (removing boilerplate from RSS feeds) and the UI (formatting raw text for readability).

Key Functions

- **`strip_html_tags(text)`**: Removes all HTML tags and entities while collapsing multiple whitespaces into a single space app/utils/html_utils.py:18-40. It utilizes `ftfy` to repair common encoding "mojibake" (e.g., converting `participaciÃ³n` back to `participación`) app/utils/html_utils.py:38-40.
- **`format_paragraphs(text, sentences_per_paragraph)`**: Converts plain block text into HTML `<p>` tags app/utils/html_utils.py:43-71. It preserves existing double-newline paragraph breaks but will automatically split long blocks of text every N sentences (default 4) to improve UI readability app/utils/html_utils.py:63-70.

Data Flow: Ingestion to UI

The following diagram illustrates how `html_utils` processes data from external sources into the `News` model and eventually to the News Studio UI.

Diagram: HTML Processing Pipeline

```
graph TD
    subgraph "External Source"
        RSS["RSS Entry (Summary/Content)"]
    end
```

```
    subgraph "app/utils/html_utils.py"
        STRIP["strip_html_tags()"]
        FORMAT["format_paragraphs()"]
    end

    subgraph "Storage & Display"
        DB[("News.raw_summary")]
        UI["news_detail.html"]
    end

    RSS -->|"Raw HTML"| STRIP
    STRIP -->|"Clean Text"| DB
    DB -->|"Plain Text"| FORMAT
    FORMAT -->|"HTML Paragraphs"| UI
```

Sources: app/utils/html_utils.py:1-72, app/templates/news_detail.html:31-36

Text Compaction and Extraction

The `app/utils/text_compaction.py` module provides logic for "semantic compression"—reducing text size without cutting mid-word or mid-sentence. This is primarily used by the `IGDraft` generator to fit content into Instagram's character limits.

Implementation Details

- **Sentence Splitting**: `split_sentences` uses regex to break text on punctuation (`.`, `!`, `?`) followed by whitespace, ensuring trailing punctuation is preserved app/utils/text_compaction.py:11-17.
- **Safe Truncation**: `truncate_at_sentence` attempts to cut text at the last sentence boundary within a character limit. If no boundary exists, it falls back to a word boundary, specifically avoiding "dangling" prepositions like "de", "con", or "para" app/utils/text_compaction.py:20-44.
- **Key Sentence Extraction**: `extract_key_sentences` (and its alias `extract_bullets`) selects complete sentences that fit within a limit (usually 110 chars). It prioritizes sentences containing numbers (data points) and the news title app/utils/text_compaction.py:47-95.

- **Caption Composition**: `compose_caption_blocks` assembles various text segments (hook, bullets, CTA, source) into a final string. If the result exceeds the `max_total` (default 900), it iteratively shortens the middle sections (bullets and "lectura") using sentence-aware truncation app/utils/text_compaction.py:109-146.

Logic Entity Mapping

This diagram maps natural language requirements (e.g., "Don't cut words") to the specific code entities implementing them.

Diagram: Compaction Logic Mapping

```
graph LR
    subgraph "Natural Language Requirements"
        R1["Split into sentences"]
        R2["Find data-rich points"]
        R3["Fit Instagram limits"]
        R4["Avoid ugly cuts"]
    end

    subgraph "app/utils/text_compaction.py"
        F1["split_sentences()"]
        F2["extract_key_sentences()"]
        F3["compose_caption_blocks()"]
        F4["truncate_at_sentence()"]
    end

    R1 --- F1
    R2 --- F2
    R3 --- F3
    R4 --- F4
```

Sources: app/utils/text_compaction.py:1-147

RSS Parsing Utilities

The `app/utils/rss_utils.py` module contains specialized helpers for handling the `feedparser` objects used during news ingestion.

`parse_published_date(entry)`

This function handles the inconsistency of date fields in RSS feeds app/utils/rss_utils.py:11-26. 1. Checks for `published_parsed`. 2. Falls back to `updated_parsed`. 3. If both are missing or invalid, it returns the current UTC time (`datetime.now(timezone.utc)`). 4. All returned dates are normalized to the UTC timezone app/utils/rss_utils.py:18, 23, 26.

Sources: app/utils/rss_utils.py:1-27

Utility Summary Table

Module	Primary Function	Input Type	Use Case
`html_utils`	`strip_html_tags`	String (HTML)	Cleaning RSS content during ingestion.
`html_utils`	`format_paragraphs`	String (Text)	Displaying summaries in the News Studio UI.
`text_compaction`	`extract_bullets`	String (Text)	Identifying key facts for Instagram slides.
`text_compaction`	`truncate_at_sentence`	String (Text)	Shortening captions to meet character limits.
`rss_utils`	`parse_published_date`	`feedparser` Entry	Normalizing timestamps across different news sources.

Sources: app/utils/html_utils.py:1-72, app/utils/text_compaction.py:1-147, app/utils/rss_utils.py:1-27

API Reference

The Althara News Service exposes a RESTful API built with FastAPI. The API is divided into several functional groups (routers) that handle public news consumption, administrative ingestion tasks, brand-specific transformations, and social media draft management.

API Router Organization

The service uses a modular routing structure to separate concerns between general news access, administrative operations, and specific brand workflows (Althara for real estate and Oxono for tech).

Natural Language to Code Entity Mapping: Router Structure

```
graph TD
    subgraph "API Entry Points"
        R1["/api/news"]
        R2["/api/admin"]
        R3["/api/tech/admin"]
        R4["/api/ig"]
    end

    subgraph "Code Entities (Routers)"
        R1 --> NewsRouter["app/routers/news.py"]
        R2 --> AdminRouter["app/routers/admin.py"]
        R3 --> TechAdminRouter["app/routers/tech_admin.py"]
        R4 --> IGRouter["app/routers/ig_drafts.py"]
    end

    subgraph "Logic Handlers"
        NewsRouter --> NR_Logic["News CRUD & Filtering"]
        AdminRouter --> AR_Logic["Real Estate Ingestion & Adaptation"]
        TechAdminRouter --> TAR_Logic["Tech Ingestion (Oxono)"]
        IGRouter --> IGR_Logic["Instagram Draft Lifecycle"]
    end
```

Sources: app/routers/news.py:19-20, app/routers/admin.py:19-20, app/routers/tech_admin.py:20-21, app/routers/ig_drafts.py:24-25

News API

The `/api/news` router provides public-facing endpoints for consuming news content. It supports complex filtering by category, date, and domain, as well as keyword searching.

- **Core Endpoints**:
  - `GET /health`: Basic service status check.
  - `POST /news`: Manually create a news item (protected by guardrails).
  - `GET /news`: List news items with pagination and filters (category, domain, relevance).
  - `GET /news/{id}`: Retrieve full details for a specific news item.
- **Key Schemas**: Uses `NewsRead` for output and `PaginatedResponse` for list results.

For details, see News API.

Sources: app/routers/news.py:21-146, app/schemas/news.py:1-13

Admin API (Real Estate)

The `/api/admin` router manages the ingestion and adaptation pipeline for the **Althara** (Real Estate) brand. It allows administrators to trigger RSS scraping and content transformation into the Althara brand voice.

- **Core Endpoints**:
  - `POST /ingest`: Triggers RSS ingestion based on `REAL_ESTATE_RSS_LIMIT_PER_SOURCE`.
  - `POST /adapt-pending`: Processes news missing summaries or structured content.
  - `POST /ingest-and-adapt`: A full-pipeline endpoint for automated cron jobs.
  - `DELETE /clean`: Domain-specific cleanup of news records.
- **Automation**: Supports `AUTO_GENERATE_IG_AFTER_INGEST` flags to streamline social media workflows.

For details, see Admin API (Real Estate).

Sources: app/routers/admin.py:22-182, app/config.py:33-45

Tech Admin API (Oxono)

The `/api/tech/admin` router mirrors the administrative capabilities of the real estate side but is specialized for the **Oxono** (Tech) brand. It uses distinct RSS sources and tech-specific guardrails.

- **Core Endpoints**:
  - `POST /ingest`: Fetches news from tech-specific sources (e.g., Wired, Genbeta).
  - `POST /ingest-and-generate`: Executes ingestion and immediately generates tech-focused Instagram drafts.

For details, see Tech Admin API (Oxono).

Sources: app/routers/tech_admin.py:23-85

Instagram Drafts API

The `/api/ig` router manages the lifecycle of social media content. It handles the generation of carousel slides, captions, and the transition of drafts through various approval states.

- **Draft Lifecycle**: `DRAFT` → `NEEDS_REVIEW` → `APPROVED` → `PUBLISHED`.
- **Core Endpoints**:
  - `POST /news/{id}/generate`: Creates a new draft from a news item.
  - `POST /{id}/approve`: Moves a draft to the approved state.
  - `POST /{id}/publish`: Marks a draft as published and updates the parent news item.
- **Security**: Requires `X-INGEST-TOKEN` for sensitive operations.

For details, see Instagram Drafts API.

Sources: app/routers/ig_drafts.py:27-250, app/models/ig_draft.py:10-25

Request Flow and Data Transformation

The following diagram illustrates how a request flows through the API routers to interact with the underlying data models and adaptation logic.

Code Entity Space: API Request Pipeline

```
sequenceDiagram
    participant Client
    participant Router as "FastAPI Router (e.g., admin.py)"
    participant Logic as "Adapters (news_adapter.py)"
    participant DB as "SQLAlchemy (News Model)"

    Client->>Router: POST /api/admin/ingest-and-adapt
    Router->>DB: Check for News.althara_content IS NULL
    DB-->>Router: Return pending news items
    loop For each News item
        Router->>Logic: build_all_content_structured()
        Logic-->>Router: althara_summary, instagram_post, structured_content
        Router->>DB: Update News record
    end
    Router->>DB: commit()
    Router-->>Client: JSON {status: "ok", adapted: X}
```

Sources: app/routers/admin.py:102-182, app/adapters/news_adapter.py:20-50, app/models/news.py:15-50

News API

The **News API** provides the primary interface for managing and retrieving news articles within the Althara News Service. It handles the lifecycle of news records, from manual creation with guardrail validation to complex filtered queries for both the Althara (real estate) and Oxono (tech) brands.

Overview of Endpoints

The endpoints are defined in `app/routers/news.py` and interact with the `News` SQLAlchemy model.

Method	Endpoint	Description
`GET`	`/health`	Simple health check to verify service availability.
`POST`	`/news`	Manually create a news item with sector-specific relevance validation.
`GET`	`/news`	List news items with extensive filtering, search, and pagination.
`GET`	`/news/{id}`	Retrieve full details of a specific news article by its UUID.

Sources: app/routers/news.py:21-146

Data Models and Schemas

The API uses Pydantic schemas defined in `app/schemas/news.py` to enforce data integrity and provide automatic documentation.

News Schemas

- **`NewsCreate`**: Used for `POST` requests. It inherits from `NewsBase` and includes fields like `title`, `source`, `url`, `category`, and `domain`. app/schemas/news.py:23-25

- `NewsRead` : Used for response serialization. It extends the base model with system-generated fields: `id` (UUID), `created_at` , and `updated_at` . app/schemas/news.py:27-34

Pagination Schema

All list responses are wrapped in a `PaginatedResponse` generic class, ensuring a consistent structure for frontend consumers. app/schemas/news.py:39-48

```
classDiagram
    class NewsBase {
        +str title
        +str source
        +str url
        +datetime published_at
        +str category
        +str domain
        +int relevance_score
    }
    class NewsCreate {
    }
    class NewsRead {
        +UUID id
        +datetime created_at
        +datetime updated_at
    }
    class PaginatedResponse~T~ {
        +List~T~ items
        +int total
        +int limit
        +int offset
        +bool has_more
    }
    NewsBase <|-- NewsCreate
    NewsBase <|-- NewsRead
    PaginatedResponse o-- NewsRead
```

Sources: app/schemas/news.py:6-49

News Creation and Guardrails

When a news item is created via `POST /news`, the system executes a validation step using the `passes_guardrails` utility. This ensures that only relevant content (e.g., housing, mortgages, construction) is persisted to the database.

- **Validation Logic**: It checks the `title` and `raw_summary` against `DENY_KEYWORDS` and `ALLOW_KEYWORDS`. app/routers/news.py:32-35
- **Failure State**: If the content fails validation, the API returns a `400 Bad Request` with a detail message explaining the sector relevance requirements. app/routers/news.py:36-39

Data Flow: Manual Ingestion

```
sequenceDiagram
    participant Client
    participant Router as "routers/news.py (create_news)"
    participant Guard as "utils/guardrails.py (passes_guardrails)"
    participant DB as "SQLAlchemy (News Model)"

    Client->>Router: POST /api/news (NewsCreate)
    Router->>Guard: Validate (title, summary, url)
    alt Fails Guardrails
        Guard-->>Router: False
        Router-->>Client: 400 Bad Request
    else Passes Guardrails
        Guard-->>Router: True
        Router->>DB: session.add(new_news)
        DB-->>Router: Commit & Refresh
        Router-->>Client: 201 Created (NewsRead)
    end
```

Sources: app/routers/news.py:25-45, app/utils/guardrails.py:15-15

Filtering and Pagination

The `GET /news` endpoint supports a wide array of query parameters to serve different parts of the application, such as the News Studio UI or the Althara web portal.

Query Parameters

- **Domain Filtering**: The `domain` parameter defaults to `real_estate` for backward compatibility but can be set to `tech` or `all`. app/routers/news.py:89-94
- **Draft Status**: The `only_with_draft` flag uses an `exists()` subquery to filter news that has associated Instagram drafts in the `ig_drafts` table. app/routers/news.py:96-98
- **Search and Dates**: Supports case-insensitive title search (`q`) via SQLAlchemy `ilike` and date range filtering on `published_at`. app/routers/news.py:76-83
- **Ordering**: Users can sort by `published_at` (default) or `relevance_score`. Sorting by relevance uses `.nullslast()` to ensure unrated news appears at the end. app/routers/news.py:104

Implementation Detail: Pagination Logic

The router calculates `has_more` by comparing `offset + limit` against the total count of items matching the criteria. app/routers/news.py:114

Sources: app/routers/news.py:48-122

Error Handling

The API implements structured error handling to prevent leaking database internals while providing useful logs for developers. - **SQLAlchemyError**: Caught and logged with `exc_info=True`. Returns a `500 Internal Server Error` with a generic "Error accessing database" message. app/routers/news.py:123-128 - **404 Not Found**: Specifically raised in `get_news` if a UUID does not match any record. app/routers/news.py:143-144

Sources: app/routers/news.py:123-134

Admin API (Real Estate)

The Admin API provides a set of protected endpoints designed to manage the news lifecycle for the Real Estate domain (Althara). These endpoints handle bulk operations including multi-source RSS ingestion, content adaptation via LLM-driven transformations, and database maintenance.

Overview of Admin Endpoints

The admin logic is centralized in `routers/admin.py`. These routes are typically invoked by automated cron jobs or via the News Studio UI to keep the Althara portal updated with fresh, brand-aligned content.

Ingestion and Adaptation Pipeline

The pipeline follows a multi-stage flow: 1. **Ingest**: Fetch raw data from external RSS feeds. 2. **Adapt**: Transform raw HTML and summaries into structured JSON and brand-specific narratives. 3. **Generate**: Create social media drafts (Instagram) based on the adapted content.

Data Flow: Ingest to Adapt

The following diagram illustrates the transition from raw external data to structured internal models within the Admin API context.

Diagram: Pipeline Data Flow

```
graph TD
    subgraph "External Space"
        RSS["RSS Sources"]
    end

    subgraph "Code Entity Space (routers/admin.py)"
        ING["POST /ingest"]
        ADAPT["POST /adapt-pending"]
        FULL["POST /ingest-and-adapt"]
    end
```

```
    subgraph "Processing Logic"
        RI["rss_ingestor.py: ingest_rss_sources"]
        NA["news_adapter.py: build_all_content_structured"]
        IG["ig_adapter.py: generate_ig_draft"]
    end

    subgraph "Persistence (SQLAlchemy)"
        DB_NEWS[("News Table")]
        DB_DRAFT[("IGDraft Table")]
    end

    RSS -->|Fetch| RI
    RI -->|Create| DB_NEWS
    ING --> RI
    ADAPT --> NA
    NA -->|Update althara_content| DB_NEWS
    FULL --> RI
    FULL --> NA
    FULL -->|Optional| IG
    IG -->|Create| DB_DRAFT
```

Sources: app/routers/admin.py:22-182, app/ingestion/rss_ingestor.py:1-20, app/adapters/news_adapter.py:1-30

Core Endpoints

1. Ingestion Endpoints

- **`POST /api/admin/ingest`**: Triggers the ingestion of real estate news. It retrieves the maximum items allowed per source from `settings.REAL_ESTATE_RSS_LIMIT_PER_SOURCE` app/routers/admin.py:33-34.
- **`POST /api/admin/ingest/rss`**: An alias for the `/ingest` endpoint app/routers/admin.py:38-46.

2. Adaptation Endpoints

- **`POST /api/admin/adapt-pending`**: Scans the `News` table for records where `althara_summary`, `instagram_post`, or `althara_content` are `None` app/routers/

admin.py:60-64. It then calls `build_all_content_structured` to populate these fields using the Althara brand voice app/routers/admin.py:72-79.

3. Integrated Pipeline

- **`POST /api/admin/ingest-and-adapt`**: A high-level orchestration endpoint designed for remote cron jobs. It performs ingestion first, then adaptation, and optionally generates Instagram drafts if the `generate_ig` query parameter is `True` and `AUTO_GENERATE_IG_AFTER_INGEST` is enabled in configuration app/routers/admin.py:102-153.

4. Maintenance Endpoints

- **`DELETE /api/admin/clean`**: Removes news items filtered by domain. It supports `real_estate`, `tech`, or `all` app/routers/admin.py:185-199.
- **`DELETE /api/admin/clean-all`**: A shortcut to delete every record in the `News` table. Due to foreign key constraints with `ondelete="CASCADE"`, this also wipes all associated `IGDraft` records app/routers/admin.py:215-221.
- **`DELETE /api/admin/news/{id}`**: Deletes a specific news item by its UUID app/routers/admin.py:231-245.

Sources: app/routers/admin.py:22-245, app/config.py:29-32

Configuration Flags

The behavior of these endpoints is governed by the `Settings` class in `app/config.py`.

Flag	Type	Description
`REAL_ESTATE_RSS_LIMIT_PER_SOURCE`	`int`	Max items to fetch from each RSS feed during `/ingest` (Default: 10).
`AUTO_GENERATE_IG_AFTER_INGEST`	`bool`	Global toggle for automatic Instagram draft creation during full pipeline runs.
`INGEST_TOKEN`	`str`	

Flag	Type	Description
		Security token required in headers for authenticated admin requests.

Sources: app/config.py:25-32, app/routers/admin.py:116-153

Technical Implementation Details

Dependency Injection

All admin routes utilize the `get_db` dependency to acquire an `AsyncSession` for database interactions app/routers/admin.py:11.

Logic Entity Mapping

The following diagram maps the API routes to the specific functions and models they manipulate.

Diagram: Route to Entity Mapping

```
classDiagram
    class AdminRouter {
        +ingest_news()
        +adapt_pending_news()
        +ingest_and_adapt()
        +clean_news_by_domain()
    }

    class IngestionLogic {
        +ingest_rss_sources(db, limit)
    }

    class AdaptationLogic {
        +build_all_content_structured(title, summary, ...)
    }

    class SocialLogic {
        +generate_ig_draft(news, brand)
    }
```

```
    class Models {
        +News
        +IGDraft
    }

    AdminRouter --> IngestionLogic : calls
    AdminRouter --> AdaptationLogic : calls
    AdminRouter --> SocialLogic : calls
    AdminRouter --> Models : queries/updates
```

Sources: app/routers/admin.py:1-17, app/database.py:1-20

Error Handling

The endpoints implement `try-except` blocks within loops (e.g., during adaptation or IG generation) to ensure that a failure in processing one news item does not halt the entire batch process app/routers/admin.py:70-91, app/routers/admin.py:165-171.

Transaction Management

For bulk operations like `adapt-pending`, the `db.commit()` is only called if the `adapted_count` is greater than zero, optimizing database writes app/routers/admin.py:93-94.

Sources: app/routers/admin.py:93-99, app/routers/admin.py:173-175

Tech Admin API (Oxono)

The Tech Admin API provides administrative endpoints specifically for the **Oxono** brand (tech domain). These endpoints mirror the real-estate ingestion flow but utilize tech-specific configurations, guardrails, and classification logic to populate the news service with technology-related content.

Purpose and Scope

The `tech_admin` router handles the automated discovery and processing of tech news. It serves as the entry point for cron jobs or manual triggers to: 1. Fetch news from tech-focused RSS feeds. 2. Filter content using tech-specific guardrails. 3. Classify news into tech categories and calculate relevance scores. 4. Optionally trigger the automated generation of Instagram drafts using the Oxono brand voice.

Data Flow and Implementation

The ingestion process is orchestrated by the `app.routers.tech_admin` module, which delegates the heavy lifting to the `tech_rss_ingestor`.

Ingestion Logic Flow

When an ingestion request is received, the system follows a structured pipeline:

1. **Source Retrieval**: Iterates through `TECH_RSS_SOURCES` defined in `constants_tech.py`.
2. **HTTP Fetching**: Uses `httpx.AsyncClient` to retrieve the XML feed.
3. **Guardrails**: Applies `passes_guardrails()` using tech-specific `DENY_KEYWORDS` and `ALLOW_KEYWORDS`.
4. **Deduplication**: Checks the database for existing URLs to prevent duplicate entries.

5. **Classification**: Calls `infer_category()` and `compute_relevance_score()` to determine the value of the news item.
6. **Persistence**: Saves the record with `domain="tech"`.

Technical Architecture Diagram

The following diagram illustrates the relationship between the API router, the ingestor service, and the supporting tech-specific logic.

Tech Ingestion Architecture

```
graph TD
    subgraph "Code Entity Space"
        R["routers/tech_admin.py"]
        I["ingestion/tech_rss_ingestor.py"]
        C["tech/classifier.py"]
        G["utils/guardrails.py"]
        M["models/news.py:News"]
        D["models/ig_draft.py:IGDraft"]
    end

    subgraph "Natural Language Space"
        API["POST /api/tech/admin/ingest"]
        RSS["Tech RSS Sources (WIRED, etc)"]
        DB[("PostgreSQL")]
    end

    API --> R
    R -- "calls" --> I
    I -- "fetches" --> RSS
    I -- "validates" --> G
    I -- "categorizes" --> C
    I -- "persists" --> M
    M -- "stored in" --> DB
    R -- "generates drafts" --> D
```

Sources: app/routers/tech_admin.py:1-76, app/ingestion/tech_rss_ingestor.py:25-121

API Endpoints

POST `/api/tech/admin/ingest`

Triggers the ingestion of tech news from RSS sources. It uses the `TECH_RSS_LIMIT_PER_SOURCE` setting to limit the number of items processed per feed.

- **Function**: `ingest_tech` app/routers/tech_admin.py:18-24
- **Key Internal Call**: `ingest_tech_rss_sources(db, max_items_per_source=...)` app/ingestion/tech_rss_ingestor.py:25-121

POST `/api/tech/admin/ingest-and-generate`

A compound endpoint that performs ingestion and then automatically generates Instagram drafts for the most relevant news items that do not yet have a draft.

- **Function**: `ingest_and_generate` app/routers/tech_admin.py:27-75
- **Logic**:
  1. Runs the standard ingestion flow. app/routers/tech_admin.py:32-34
  2. Queries for tech news without drafts, ordered by `relevance_score` DESC. app/routers/tech_admin.py:46-55
  3. Calls `generate_ig_draft()` with `brand="oxono"`. app/routers/tech_admin.py:60
  4. Commits new `IGDraft` records to the database. app/routers/tech_admin.py:68

Sources: app/routers/tech_admin.py:18-76

Ingestion Pipeline Detail

The `ingest_tech_rss_sources` function is the core of the tech domain's data acquisition. Unlike the real estate ingestor, it is tailored for the Oxono brand's requirements.

Process Logic

Step	Logic / Function	File Reference
Source Loop	Iterates over `TECH_RSS_SOURCES`	app/ingestion/tech_rss_ingestor.py:35
Guardrails	`passes_guardrails(title, DENY_KEYWORDS, ...)`	app/ingestion/tech_rss_ingestor.py:74-78
Deduplication	`select(News).where(News.url == link)`	app/ingestion/tech_rss_ingestor.py:81-84
Classification	`infer_category(title, summary)`	app/ingestion/tech_rss_ingestor.py:87
Scoring	`compute_relevance_score(...)`	app/ingestion/tech_rss_ingestor.py:89-91
Minimum Quality	Filters if `relevance_score < MIN_SCORE`	app/ingestion/tech_rss_ingestor.py:93-94

Code Mapping: Ingestion to Persistence

This diagram bridges the functional steps of the `ingest_tech_rss_sources` function to the specific code entities and data models involved.

Ingestion Function Internal Flow

```
flowchart LR
    Start(["ingest_tech_rss_sources"]) --> Loop["For source in TECH_RSS_SOURCES"]
    Loop --> Fetch["httpx.get(feed_url)"]
    Fetch --> Parse["feedparser.parse()"]
    Parse --> Entry["For entry in feed.entries"]

    subgraph Validation ["Validation & Scoring"]
        Entry --> Guard["passes_guardrails()"]
        Guard --> Score["compute_relevance_score()"]
    end

    Score --> Filter{"Score >= MIN_SCORE?"}
    Filter -- "Yes" --> Model["Create News(domain='tech')"]
    Model --> DB[("session.add(new_news)")]
    DB --> Commit["session.commit()"]
```

```
    Filter -- "No" --> Entry
```

Sources: app/ingestion/tech_rss_ingestor.py:25-121, app/constants_tech.py:12-18

Key Configuration Constants

The behavior of these endpoints is heavily influenced by constants and settings:

- `TECH_RSS_SOURCES`: A list of dictionaries containing `name`, `url`, `source` (label), and `default_category`. app/constants_tech.py:13
- `TECH_RSS_LIMIT_PER_SOURCE`: Configurable via environment variables, determines how many items to pull per source. app/routers/tech_admin.py:22
- `AUTO_GENERATE_IG_AFTER_INGEST`: A boolean flag in `settings`. If false, the `ingest-and-generate` endpoint skips the draft creation phase. app/routers/tech_admin.py:37
- `MIN_SCORE`: The threshold for relevance; items scoring lower are discarded during ingestion. app/ingestion/tech_rss_ingestor.py:93

Sources: app/ingestion/tech_rss_ingestor.py:12-18, app/routers/tech_admin.py:22-37

Instagram Drafts API

The Instagram Drafts API provides a set of endpoints for managing the lifecycle of social media content derived from ingested news. It handles the generation of carousel slides, captions, and hashtags, while maintaining a state machine for editorial review and publication.

Overview and Lifecycle

The API facilitates the transition of news content into structured Instagram posts. Every `IGDraft` is linked to a parent `News` record and follows a specific status lifecycle to ensure quality control before publication.

IGDraft Status Lifecycle

The `status` field in the `IGDraft` model tracks the progress of a draft app/models/ig_draft.py:23: 1. **DRAFT**: The initial state upon generation. 2. **NEEDS_REVIEW**: Set when a draft requires manual intervention. 3. **APPROVED**: Marked as ready for publication by an editor. 4. **PUBLISHED**: Final state indicating the content has been posted.

Authentication

Endpoints that modify data (POST, PATCH) are protected by the `_require_token` dependency app/routers/ig_drafts.py:20-25. This check validates the `X-INGEST-TOKEN` HTTP header against the `INGEST_TOKEN` value defined in the system settings app/config.py.

Sources: - app/models/ig_draft.py:9-31 - app/routers/ig_drafts.py:20-25

Data Flow: Generation to Publication

The following diagram illustrates how a `News` entity is transformed into an `IGDraft` and moved through the lifecycle via the `ig_drafts.py` router.

Content Transformation Flow

```
sequenceDiagram
    participant Client
    participant Router as routers/ig_drafts.py
    participant Adapter as adapters/ig_adapter.py
    participant DB as SQLAlchemy (ig_drafts table)

    Client->>Router: POST /news/{id}/generate
    Router->>DB: select(News).where(id==news_id)
    Router->>Adapter: generate_ig_draft(news, tone, brand)
    Adapter-->>Router: draft_data (dict)
    Router->>DB: INSERT/UPDATE IGDraft
    DB-->>Router: IGDraft Object
    Router-->>Client: 200 OK (IGDraftRead)

    Client->>Router: POST /{id}/approve
    Router->>DB: UPDATE ig_drafts SET status='APPROVED'
    Router-->>Client: 200 OK

    Client->>Router: POST /{id}/publish
    Router->>DB: UPDATE ig_drafts SET status='PUBLISHED'
    Router->>DB: UPDATE news SET used_in_social=true
    Router-->>Client: 200 OK
```

Sources: - app/routers/ig_drafts.py:28-65 - app/routers/ig_drafts.py:155-197

Schema and Models

The system uses Pydantic schemas for data validation and SQLAlchemy models for persistence.

Database Entity: `IGDraft`

The `IGDraft` model stores the structured content for Instagram. Key fields include `carousel_slides` (stored as JSONB) and `variant_of_id` for supporting multiple versions of the same news item.

Field	Type	Description
`id`	UUID	Primary Key app/models/ig_draft.py:12

Field	Type	Description
`news_id`	UUID	Foreign Key to `news.id` app/models/ig_draft.py:13
`carousel_slides`	JSONB	List of `{title, body}` objects app/models/ig_draft.py:15
`hashtags`	JSONB	Array of strings app/models/ig_draft.py:17
`status`	Text	DRAFT, NEEDS_REVIEW, APPROVED, or PUBLISHED app/models/ig_draft.py:23
`variant_of_id`	UUID	Self-referencing FK for variants app/models/ig_draft.py:25

API Entities: Pydantic Schemas

- **`IGDraftCreate`**: Used for internal creation logic app/schemas/ig_draft.py:26-27.
- **`IGDraftUpdate`**: Allows partial updates to fields like `hook`, `caption`, or `status` via `PATCH` app/schemas/ig_draft.py:30-41.
- **`IGDraftRead`**: The public-facing representation, including timestamps and IDs app/schemas/ig_draft.py:44-52.

Sources: - app/models/ig_draft.py:9-31 - app/schemas/ig_draft.py:1-53

Endpoint Reference

Generation and Variants

- **`POST /api/ig/news/{news_id}/generate`**: Triggers the `generate_ig_draft` function. If a primary draft (where `variant_of_id` is null) already exists, it updates it; otherwise, it creates a new one app/routers/ig_drafts.py:28-65.
- **`POST /api/ig/news/{news_id}/variants`**: Generates multiple alternative versions using `generate_variants`. These are linked to the news item but treated as distinct options for the editor app/routers/ig_drafts.py:68-98.

CRUD and Management

- **`GET /api/ig/`**: Lists drafts with support for `domain` and `status` filtering, including pagination app/routers/ig_drafts.py:101-119.

- `GET /api/ig/{draft_id}` : Retrieves a specific draft by its UUID app/routers/ig_drafts.py:122-130.
- `PATCH /api/ig/{draft_id}` : Updates specific fields. Commonly used by the News Studio UI to save manual edits to captions or slides app/routers/ig_drafts.py:133-152.

Workflow Transitions

- `POST /api/ig/{draft_id}/approve` : Explicitly sets status to `APPROVED` app/routers/ig_drafts.py:155-170.
- `POST /api/ig/{draft_id}/publish` : Sets status to `PUBLISHED` . If the `mark_news_used` query parameter is true (default), it also updates the parent `News` record's `used_in_social` flag to `True` app/routers/ig_drafts.py:173-197.

Code Entity Mapping

This diagram maps the API routes to their corresponding logic and data structures.

```
graph TD
    subgraph "FastAPI Router [routers/ig_drafts.py]"
        R_GEN["POST /news/{id}/generate"]
        R_VAR["POST /news/{id}/variants"]
        R_PATCH["PATCH /{id}"]
        R_PUB["POST /{id}/publish"]
    end

    subgraph "Logic & Adapters"
        A_GEN["generate_ig_draft() [ig_adapter.py]"]
        A_VAR["generate_variants() [ig_adapter.py]"]
    end

    subgraph "Persistence [models/ig_draft.py]"
        M_DRAFT["IGDraft (ORM Model)"]
        M_NEWS["News (ORM Model)"]
    end

    R_GEN --> A_GEN
    R_VAR --> A_VAR
    A_GEN --> M_DRAFT
    A_VAR --> M_DRAFT
    R_PATCH --> M_DRAFT
    R_PUB --> M_DRAFT
    R_PUB --> M_NEWS
```

Sources: - app/routers/ig_drafts.py:1-198 - app/models/ig_draft.py:9-31 - app/adapters/ig_adapter.py:1-20 (implied by imports in router)

News Studio UI

The **News Studio UI** is a dedicated internal web interface designed for the marketing team to manage the lifecycle of news content. It provides a visual layer over the news ingestion and adaptation pipeline, allowing users to filter news by relevance, edit AI-generated Instagram drafts, and oversee the transition from raw RSS data to approved social media posts.

UI Architecture Overview

The interface is built using **FastAPI**'s `Jinja2Templates` engine app/routers/ui.py:8-27. It serves server-side rendered (SSR) HTML pages that interact with the backend via standard HTTP GET requests for navigation and asynchronous `fetch` calls (POST/PATCH) for data operations like saving drafts or triggering ingestion app/templates/portal.html:122-128.

The UI is divided into four primary views: 1. **Brand Selector**: Entry point to choose between Althara (Real Estate) and Oxono (Tech). 2. **Portal (Inbox/Drafts/Approved)**: A multi-tab dashboard for news management. 3. **News Detail**: A deep-dive view of a single news item and its associated drafts. 4. **IG Draft Editor**: A specialized tool for fine-tuning Instagram carousels and captions.

System Mapping: UI to Code Entities

The following diagram maps the visual components of the News Studio to their corresponding backend routes and database models.

UI Component to Code Mapping

```
graph TD
    subgraph "Browser (Natural Language Space)"
        "Brand Selector" --> "Portal Dashboard"
        "Portal Dashboard" --> "News Detail View"
        "News Detail View" --> "Instagram Editor"
    end

    subgraph "Backend (Code Entity Space)"
```

```
        "Brand Selector" --- UI_SELECTOR["ui_selector() route<br/>app/routers/ui.py"]
        "Portal Dashboard" --- UI_PORTAL["ui_portal() route<br/>app/routers/ui.py"]
        "News Detail View" --- UI_NEWS_DETAIL["ui_news_detail() route<br/>app/routers/ui.py"]
        "Instagram Editor" --- UI_DRAFT_EDITOR["ui_draft_editor() route<br/>app/routers/ui.py"]

        UI_PORTAL --> NEWS_MODEL["News Model<br/>app/models/news.py"]
        UI_DRAFT_EDITOR --> DRAFT_MODEL["IGDraft Model<br/>app/models/ig_draft.py"]
    end

    UI_SELECTOR -.-> SELECTOR_HTML["selector.html"]
    UI_PORTAL -.-> PORTAL_HTML["portal.html"]
    UI_DRAFT_EDITOR -.-> EDITOR_HTML["draft_editor.html"]
```

Sources: app/routers/ui.py:35-181, app/templates/selector.html:1-14, app/templates/portal.html:1-99, app/templates/draft_editor.html:1-138

Core Navigation and Branding

The UI supports a multi-brand architecture defined in `app/brands.py`. Upon entering the `/ui` route, users are presented with brand cards app/templates/selector.html:7-13. Selecting a brand sets the context for all subsequent views, including the visual theme (CSS variables) and the data domain (e.g., `real_estate` for Althara or `tech` for Oxono) app/routers/ui.py:59-66.

- **Brand Switcher**: A global dropdown in the header allows switching brands while maintaining the current view app/templates/portal.html:9-15.
- **Domain Filtering**: The UI automatically filters the `News` table based on the `domain` associated with the active brand app/routers/ui.py:66-67.

For details on how routes handle brand-specific logic, see UI Routing and Authentication.

Portal and News Management

The **Portal** (`/ui/{brand}`) is the central hub for the marketing team. It organizes news into three logical tabs based on their status and the existence of Instagram drafts app/routers/ui.py:61-66:

Tab	Logic	Use Case
Inbox	All news for the domain.	General browsing and initial discovery.
Drafts	News with `IGDraft` in `DRAFT` status.	Active editing and refinement.
Approved	News with `IGDraft` in `APPROVED` or `PUBLISHED` status.	History and finalized content.

The portal includes advanced filtering by category, search queries, and sorting by `relevance_score` or `published_at` app/templates/portal.html:25-54. It also features a "Generate more news" button that triggers the full ingestion and adaptation pipeline app/templates/portal.html:55.

Sources: app/routers/ui.py:44-118, app/templates/portal.html:59-64

Instagram Draft Editor

The **Draft Editor** (`/ui/{brand}/draft/{draft_id}`) is a specialized environment for content creators. It provides a split-pane layout: 1. **Editor Pane**: Form fields for editing the Hook, Carousel Slides (Title/Body), Caption, Hashtags, and CTA app/templates/draft_editor.html:30-85. 2. **Preview Pane**: A real-time "Instagram-style" phone mock-up that updates as the user types, allowing them to visualize how the carousel and caption will appear on a mobile device app/templates/draft_editor.html:104-134.

Editor Workflow

```
graph LR
    "User Input" --> "JS State Management"
    "JS State Management" --> "Live Preview"
    "JS State Management" --> SAVE["saveDraft() PATCH call"]
    SAVE --> DB[("IGDraft Table")]
```

For details on the JavaScript logic, hashtag chips, and the real-time preview engine, see Templates and Frontend Logic.

Sub-Pages

- **UI Routing and Authentication**: Details on the `ui.py` router, URL parameters for filtering, and BasicAuth security.
- **Templates and Frontend Logic**: Technical breakdown of the Jinja2 templates, CSS themes, and the JavaScript-driven editor features.

Sources: app/routers/ui.py:1-182, app/templates/draft_editor.html:1-160

UI Routing and Authentication

The News Studio UI is a server-side rendered (SSR) web interface built with FastAPI and Jinja2. It serves as the internal management portal for the marketing team to browse ingested news, review generated Instagram drafts, and edit content before publishing. Routing is handled by a dedicated `ui` router, and access is protected via a Basic Authentication mechanism.

Authentication Mechanism

The system implements a simple `BasicAuth` strategy. It is controlled by the environment variables `UI_USER` and `UI_PASS` app/config.py.

basic_auth_dependency

The `basic_auth_dependency` function app/auth.py:26-37 acts as a FastAPI dependency for all UI routes. - **Bypass:** If `UI_USER` or `UI_PASS` are not set in the environment, the dependency returns immediately, allowing unauthenticated access app/auth.py:28-29. - **Validation:** It extracts the `Authorization` header and uses `_check_basic_auth` app/auth.py:11-23 to decode the Base64 credentials and compare them against the settings app/auth.py:31. - **Failure:** If credentials are missing or incorrect, it raises an `HTTP_401_UNAUTHORIZED` exception with a `WWW-Authenticate: Basic` header to trigger the browser's login prompt app/auth.py:32-36.

Data Flow: Authentication Logic

Title: Authentication Flow for UI Routes

```
graph TD
    User["User Request"] -- "GET /ui/..." --> Dep["basic_auth_dependency"]
    Dep -- "Check settings" --> Config{"UI_USER/PASS set?"}
    Config -- "No" --> Proceed["Allow Access"]
    Config -- "Yes" --> Header{"Auth Header?"}
    Header -- "No" --> Unauth["401 Unauthorized"]
    Header -- "Yes" --> Verify["_check_basic_auth"]
```

```
    Verify -- "Match" --> Proceed
    Verify -- "Mismatch" --> Unauth
```

Sources: app/auth.py:11-37, app/routers/ui.py:36

UI Router and Brand Selection

The UI router app/routers/ui.py:24 manages four primary views. It utilizes `Jinja2Templates` app/routers/ui.py:26 and a custom filter `format_paragraphs` app/routers/ui.py:27 for content rendering.

Brand Selector (`/ui`)

The entry point is the brand selector app/routers/ui.py:35-40. It displays the available brands defined in the `BRANDS` registry (Althara and Oxono) app/brands.py:14-25. This allows users to choose the domain context (Real Estate vs. Tech) they wish to manage.

Brand Portal (`/ui/{brand}`)

The portal view app/routers/ui.py:43-118 is the main dashboard for a specific brand. It supports several query parameters for filtering: - `tab` : Switches between `inbox` (all news), `drafts` (news with pending IG drafts), and `approved` (news with approved/published drafts) app/routers/ui.py:61-65. - `q` : Full-text search on news titles using SQLAlchemy `ilike` app/routers/ui.py:68-69. - `category` : Filters by domain-specific categories app/routers/ui.py:70-71. - `order_by` : Toggles between `published_at` and `relevance_score` app/routers/ui.py:76-79.

The portal logic automatically maps the URL `{brand}` to the internal `domain` (e.g., `althara` -> `real_estate`) using `get_domain_for_brand` app/routers/ui.py:59.

Code-to-Entity Mapping: Portal Query Logic

Title: UI Portal Data Retrieval

```
graph TD
    Route["ui_portal (/ui/{brand})"] -- "Calls" --> Domain["get_domain_for_brand(brand)"]
```

```
    Domain -- "Maps to" --> DB_Domain["News.domain"]
    Route -- "Constructs" --> SQL["SQLAlchemy select(News)"]
    SQL -- "Joins if tab=drafts" --> IGD["IGDraft Table"]
    SQL -- "Filter" --> Cat["News.category == category"]
    SQL -- "Order" --> Rel["News.relevance_score DESC"]
    SQL -- "Executes" --> DB[("PostgreSQL")]
    DB -- "Returns" --> NewsList["news_list"]
```

Sources: app/routers/ui.py:43-82, app/brands.py:28-31

News Detail and Draft Editor

News Detail (`/ui/{brand}/news/{news_id}`)

This view app/routers/ui.py:121-146 provides a deep dive into a specific `News` record. - It fetches the `News` object and all associated `IGDraft` records app/routers/ui.py:133-141. - It displays the full content, including the structured Althara content or raw summaries. - It provides action buttons to trigger the generation of new Instagram drafts.

Draft Editor (`/ui/{brand}/draft/{draft_id}`)

The editor app/routers/ui.py:149-181 is the final step in the content pipeline. - It loads a specific `IGDraft` and its parent `News` record app/routers/ui.py:161-169. - The template `draft_editor.html` renders the carousel slides, caption, and hashtags for manual refinement. - It allows the user to approve or publish the draft, interacting with the `/api/ig` endpoints via client-side JavaScript.

Data Flow: UI Routing

The following table summarizes the data requirements and behavior for each UI route:

Route	Function	Key Models	Logic / Filters
`/ui`	`ui_selector`	N/A	Lists `BRANDS` from `brands.py`.
`/ui/{brand}`	`ui_portal`	`News`, `IGDraft`	Filters by `domain`. Joins `IGDraft` for "drafts" and "approved" tabs.
`/ui/{brand}/news/{id}`	`ui_news_detail`	`News`, `IGDraft`	Fetches single `News` and `select(IGDraft).where(news_id==id)`.
`/ui/{brand}/draft/{id}`	`ui_draft_editor`	`IGDraft`, `News`	Fetches single `IGDraft` and its parent `News`.

Sources: app/routers/ui.py:35-181, app/brands.py:14-25

Code-to-Entity Mapping: Routing Structure

Title: UI Router Code Structure

```
classDiagram
    class UI_Router {
        +ui_selector(request)
        +ui_portal(brand, tab, q, category)
        +ui_news_detail(brand, news_id)
        +ui_draft_editor(brand, draft_id)
    }
    class Dependencies {
        +get_db()
        +basic_auth_dependency()
    }
    class Models {
        +News
        +IGDraft
    }
    UI_Router ..> Dependencies : uses
    UI_Router ..> Models : queries
```

Sources: app/routers/ui.py:24-181, app/auth.py:26-37

Templates and Frontend Logic

The News Studio UI is a server-side rendered (SSR) web interface built with **Jinja2** templates and vanilla **JavaScript**. It provides the marketing team with a specialized environment to manage news ingestion, review AI-generated summaries, and edit Instagram carousel drafts before publication.

Overview of the Template System

The frontend architecture follows a standard inheritance pattern using a base layout. Each page is brand-aware, dynamically adjusting its theme and content based on the `brand` context variable (e.g., "althara" or "oxono").

Template	Role	Key Features
`base.html`	Root Layout	Imports `studio.css`, defines global HTML structure app/templates/base.html:1-13.
`selector.html`	Brand Entry	Displays brand cards for Althara and Oxono app/templates/selector.html:7-13.
`portal.html`	Main Dashboard	Tabbed inbox/drafts/approved views with filtering app/templates/portal.html:59-64.
`news_detail.html`	News Review	Shows raw content, AI summary, and draft variants app/templates/news_detail.html:22-54.
`draft_editor.html`	IG Editor	Real-time preview, slide management, and copy-to-clipboard app/templates/draft_editor.html:29-135.

Template Data Flow

The following diagram illustrates how the FastAPI `ui.py` router populates these templates with data from the SQLAlchemy models.

UI Data Flow Diagram

```
graph TD
    subgraph "Server Space (Python/FastAPI)"
        A["ui.py (Router)"] -- "brand, news_with_drafts" --> B["portal.html"]
        A -- "news, drafts" --> C["news_detail.html"]
        A -- "draft, news" --> D["draft_editor.html"]
    end

    subgraph "Code Entity Space"
        B -- "Iteration" --> B1["News Model"]
        C -- "Iteration" --> C1["IGDraft Model"]
        D -- "Form Binding" --> D1["IGDraft Model"]
    end

    subgraph "Natural Language Space"
        B1 -- "Displays" --> B2["Inbox/Bandeja List"]
        C1 -- "Displays" --> C2["Draft Variants List"]
        D1 -- "Edits" --> D2["Carousel Slides & Caption"]
    end
```

Sources: app/templates/portal.html:72-82, app/templates/news_detail.html:45-51, app/templates/draft_editor.html:34-84

Portal Logic and State Management

The `portal.html` template serves as the primary command center. It manages three distinct states via the `tab` URL parameter: `inbox`, `drafts`, and `approved`.

Search and Filtering

Filtering is handled via a hidden drawer that modifies URL query parameters. - **Search:** Updates the `q` parameter app/templates/portal.html:16-22. - **Category Filter:** Populated dynamically from the brand's category list app/templates/portal.html:33-38. - **Relevance Sorting:** Allows ordering by `relevance_score` or `published_at` app/templates/portal.html:42-46.

Ingestion Trigger

The "Generar más noticias" button triggers a modal app/templates/portal.html:101-110. Upon confirmation, it executes an asynchronous `POST` request to either `/`

`api/admin/ingest-and-adapt` (Althara) or `/api/tech/admin/ingest-and-generate` (Oxono) app/templates/portal.html:122-129.

Sources: app/templates/portal.html:16-64, app/templates/portal.html:122-129

Instagram Draft Editor

The `draft_editor.html` is the most complex frontend component, featuring a split-pane layout with an editor on the left and a live Instagram preview on the right.

Real-time Preview Engine

The preview pane simulates the Instagram mobile UI. It uses JavaScript event listeners on the `textarea` and `input` fields to update the preview DOM elements immediately.

- **Slide Navigation:** The carousel is managed via `activeSlideIdx`. Switching tabs updates both the editor visibility and the preview text app/templates/draft_editor.html:158-167.
- **Character Counters:** Dynamic counters warn users when text exceeds Instagram's recommended limits (e.g., 110 chars for slides, 900 for captions) app/templates/draft_editor.html:52-60.

Hashtag Chips

Hashtags are managed as an array of strings. The UI provides a "chip" interface where users can add new tags or remove existing ones app/templates/draft_editor.html:65-71. - `renderHashtags()`: Iterates the `hashtags` array and generates HTML spans app/templates/draft_editor.html:183-193. - `addHashtag()`: Parses the input, removes the `#` prefix if present, and pushes to the state app/templates/draft_editor.html:195-202.

Save Logic (PATCH)

Changes are persisted via the `saveDraft()` function, which performs a `PATCH` call to `/api/ig/{draftId}`. It gathers all slide data using `getSlides()` and synchronizes

the JSON structure with the backend `IGDraft` model app/templates/draft_editor.html:147-156, app/templates/draft_editor.html:220-234.

Editor Logic Diagram

```
sequenceDiagram
    participant User
    participant Editor as "draft_editor.js"
    participant Preview as "Preview Pane"
    participant API as "/api/ig/{id} (PATCH)"

    User->>Editor: Type in Slide Body
    Editor->>Preview: updatePreview()
    Preview-->>User: Visual Update
    User->>Editor: Click "Guardar"
    Editor->>Editor: getSlides() (Aggregate JSON)
    Editor->>API: saveDraft()
    API-->>Editor: 200 OK
    Editor->>User: showToast("Guardado")
```

Sources: app/templates/draft_editor.html:147-156, app/templates/draft_editor.html:158-181, app/templates/draft_editor.html:220-234

Content Utilities

The frontend leverages backend utilities for content presentation, particularly when displaying news summaries.

HTML Cleaning and Formatting

The `html_utils.py` module provides logic to ensure ingested content is readable in the UI: - **`strip_html_tags`**: Used to clean raw RSS content and fix encoding issues (mojibake) using `ftfy` app/utils/html_utils.py:18-40. - **`format_paragraphs`**: A Jinja2 filter that converts block text into `<p>` tags by splitting sentences every N lines, ensuring long summaries don't break the UI layout app/utils/html_utils.py:43-71.

Copy-to-Clipboard Engine

In the `draft_editor.html`, a multi-option copy menu allows users to extract content for manual posting: - **Caption:** Copies the main text block. - **Carousel:** Aggregates all slide titles and bodies into a single formatted string app/templates/draft_editor.html:90-96.

Sources: app/utils/html_utils.py:18-71, app/templates/draft_editor.html:90-96, app/templates/news_detail.html:33

Data Models and Database Schema

This section provides a high-level overview of the Althara News Service data layer. The system utilizes **SQLAlchemy** as its Object-Relational Mapper (ORM) and **Alembic** for schema versioning, targeting a **PostgreSQL** database with async capabilities.

The schema is centered around two primary entities: the raw news ingested from various sources and the generated social media drafts derived from that news.

Code Entity Space to Database Mapping

The following diagram bridges the conceptual "Natural Language" space of the news pipeline to the specific code entities defined in the persistence layer.

System Entity Mapping

```
graph TD
    subgraph "Natural Language Space"
        A["Ingested Article"]
        B["Instagram Post"]
        C["Database Version"]
    end

    subgraph "Code Entity Space"
        A --> D["News [app/models/news.py]"]
        B --> E["IGDraft [app/models/ig_draft.py]"]
        C --> F["Alembic Revisions [alembic/versions/]"]
    end

    D -- "1:N Relationship" --> E
    F -- "Manages" --> D
    F -- "Manages" --> E
```

Sources: app/models/news.py:9-31, app/models/ig_draft.py:9-30

Core ORM Models

The system defines two main models that inherit from a shared `Base` class app/database.py:9.

1. News Model

The `News` model is the primary entry point for all data in the system. It stores the raw data fetched from RSS feeds or the Idealista API, as well as the structured, brand-aligned content generated by the AI adapters.

- **Key Fields**: `title`, `url`, `category`, `althara_content` (JSONB), and `domain` (used to distinguish between Althara and Oxono).
- **Relationship**: It maintains a one-to-many relationship with `IGDraft`, configured with `cascade="all, delete-orphan"` app/models/news.py:31.

For details, see News Model (#7.1).

2. IGDraft Model

The `IGDraft` model represents a social media asset generated from a specific news item. It supports versioning through a self-referential foreign key.

- **Key Fields**: `carousel_slides` (JSONB), `caption`, `status` (DRAFT, APPROVED, etc.), and `variant_of_id`.
- **Relationship**: Linked to `News` via `news_id` with a `CASCADE` delete constraint app/models/ig_draft.py:13.

For details, see IGDraft Model (#7.2).

Database Connectivity

The service uses an asynchronous engine powered by `asyncpg`. A critical utility function, `normalize_database_url`, ensures that standard PostgreSQL connection strings are converted to the `postgresql+asyncpg://` format required by the driver app/database.py:12-49.

Connection Flow

```
graph LR
    subgraph "Application Layer"
        GDB["get_db() [app/database.py]"]
    end

    subgraph "SQLAlchemy Engine"
        ASL["AsyncSessionLocal"]
        CAE["create_async_engine"]
    end

    subgraph "PostgreSQL DB"
        PDB[("Neon / Postgres")]
    end

    GDB --> ASL
    ASL --> CAE
    CAE -- "asyncpg" --> PDB
```

Sources: app/database.py:59-95

Migration History and Alembic

Database schema changes are managed via Alembic. The migration environment alembic/env.py:1-128 is configured to handle the same async connection logic as the main application to ensure consistency during deployment.

The migration chain has evolved from a simple news table to a complex schema supporting multi-domain content, relevance scoring, and detailed social media draft metadata.

- **Initial Setup**: Creation of the `news` table.
- **Expansion**: Addition of `ig_drafts` and domain-specific fields like `relevance_score`.
- **Schema Refinement**: Introduction of `JSONB` fields for structured AI content (`althara_content`) and geographical metadata (`provincia`, `poblacion`).

For details, see Database Migrations (#7.3).

Sources: - app/models/news.py:1-33 - app/models/ig_draft.py:1-31 - app/database.py:1-95 - alembic/env.py:1-128

News Model

The `News` model is the central entity of the Althara News Service. It represents a single news article ingested from external sources (RSS feeds, Idealista, etc.) and contains both the raw metadata and the transformed content used for brand-specific publishing. The model supports multi-domain logic (Real Estate vs. Tech) and serves as the parent for Instagram drafts.

1. SQLAlchemy ORM Model

The `News` class is defined in `app/models/news.py` and inherits from the `Base` declarative class. It maps to the `news` table in PostgreSQL.

Database Schema Details

The table includes fields for original source metadata, geographic data, and multiple versions of processed content (summaries and structured JSON).

Column Name	Type	Description
`id`	`UUID`	Primary key, automatically generated using `uuid.uuid4`.
`title`	`String`	The title of the news article.
`source`	`String`	The name of the source (e.g., "El País", "Idealista").
`url`	`String`	Unique URL of the article.
`published_at`	`DateTime`	Original publication timestamp (with timezone).
`category`	`String`	Classified category (e.g., "Mercado", "Vivienda").
`raw_summary`	`Text`	The initial summary extracted during ingestion.
`althara_summary`	`Text`	Narrative summary adapted to the Althara brand voice.
`instagram_post`	`Text`	Legacy field for single-block Instagram text content.

Column Name	Type	Description
`althara_content`	`JSON`	Structured content (v2.0) containing specific keys: hecho, lectura, etc.
`tags`	`Text`	Comma-separated or space-separated keywords.
`used_in_social`	`Boolean`	Flag indicating if the news has been used in a social post.
`provincia`	`String`	Geographic province associated with the news (if applicable).
`poblacion`	`String`	Geographic town/city associated with the news.
`domain`	`Text`	Brand domain: `real_estate` (Althara) or `tech` (Oxono).
`relevance_score`	`Integer`	Computed score (0-100) based on category and keywords.
`created_at`	`DateTime`	Record creation timestamp.
`updated_at`	`DateTime`	Last update timestamp, managed by `onupdate=func.now()` .

Relationships and Cascades

The model maintains a one-to-many relationship with the `IGDraft` model. * **`ig_drafts`**: Defined with `cascade="all, delete-orphan"`. If a `News` record is deleted, all associated Instagram drafts are automatically removed from the database to maintain referential integrity app/models/news.py:31-31.

Sources: * app/models/news.py:9-31 * app/database.py:5-5 (for `Base` inheritance)

2. Pydantic Schemas

The system uses Pydantic models in `app/schemas/news.py` for data validation, API request parsing, and serialization.

Class Hierarchy

1. `NewsBase` : Contains common fields shared across creation and reading operations app/schemas/news.py:6-21.
2. `NewsCreate` : Used for `POST` requests. It inherits from `NewsBase` but does not include system-generated fields like `id` or `created_at` app/schemas/news.py:23-25.
3. `NewsRead` : Used for API responses. It includes the `id` and timestamps. It enables `from_attributes = True` to allow seamless conversion from SQLAlchemy ORM objects app/schemas/news.py:27-34.

Paginated Responses

To handle large datasets in the UI and API, the `PaginatedResponse` generic model is used. It wraps a list of items with metadata app/schemas/news.py:39-48.

Sources: * app/schemas/news.py:6-48

3. Data Flow and Entity Mapping

The following diagrams illustrate how the system maps the "Natural Language" concepts of news articles into the "Code Entity Space" of the SQLAlchemy and Pydantic models.

Diagram: Entity Mapping and Relationships

This diagram shows how the `News` entity relates to its child drafts and the database layer.

```
graph TD
    subgraph "Database Layer (SQLAlchemy)"
        DB_NEWS["class News(Base)"]
        DB_IGD["class IGDraft(Base)"]
    end

    subgraph "API Layer (Pydantic)"
        S_CREATE["class NewsCreate(NewsBase)"]
        S_READ["class NewsRead(NewsBase)"]
        S_PAGINATED["class PaginatedResponse(Generic[T])"]
    end

    subgraph "Logic & Storage"
```

```
        POSTGRES[("PostgreSQL Table: news")]
    end

    DB_NEWS -- "1:N relationship (cascade delete)" --> DB_IGD
    DB_NEWS -- "maps to" --> POSTGRES
    S_READ -- "from_attributes" --> DB_NEWS
    S_PAGINATED -- "contains" --> S_READ
    S_CREATE -- "validates input for" --> DB_NEWS
```

Sources: * app/models/news.py:9-31 * app/schemas/news.py:23-48

Diagram: Content Transformation Mapping

This diagram shows how specific news attributes map to the functional areas of the codebase.

```
graph LR
    subgraph "Ingestion Space"
        URL["url"]
        TITLE["title"]
        RAW["raw_summary"]
    end

    subgraph "Code Entity: News Model"
        direction TB
        NEWS_OBJ["News Instance"]
        ATTR_STR["althara_content (JSONB)"]
        ATTR_IG["instagram_post (Text)"]
        ATTR_DOM["domain (Text)"]
    end

    subgraph "Transformation Space"
        ADAPTER["news_adapter.py"]
        IG_GEN["ig_adapter.py"]
    end

    RAW --> ADAPTER
    ADAPTER --> ATTR_STR
    ATTR_STR --> IG_GEN
    IG_GEN --> ATTR_IG
    ATTR_DOM -- "determines logic" --> ADAPTER
```

Sources: * app/models/news.py:12-29 * app/schemas/news.py:6-21

4. Implementation Details

- **JSONB Storage**: The `althara_content` field uses the `JSON` type (mapping to `JSONB` in PostgreSQL) to store structured adaptation data. This allows the system to store version 2.0 adaptation results (keys: `hecho`, `lectura`, `implicaciones`, `senales_a_vigilar`) without requiring a rigid table schema app/models/news.py:21-21.
- **Domain Filtering**: The `domain` column app/models/news.py:26-26 defaults to `real_estate` but is used extensively by routers to filter news for the Althara (real estate) vs. Oxono (tech) portals.
- **Relevance Scoring**: The `relevance_score` app/models/news.py:27-27 is an integer field that allows the API to sort news by "importance" rather than just by date.

Sources: * app/models/news.py:21-27 * app/schemas/news.py:15-21

IGDraft Model

The `IGDraft` model represents a structured Instagram post proposal derived from a News article. It serves as the primary data structure for the social media workflow, capturing carousel content, captions, and metadata required for publication.

1. SQLAlchemy Model Definition

The `IGDraft` class is defined in `app/models/ig_draft.py` and inherits from the shared `Base` declarative class. It utilizes PostgreSQL-specific features like `JSONB` for flexible content storage.

1.1 Column Specifications

The model includes the following fields:

Column	Type	Description
`id`	`UUID`	Primary key, generated via `uuid.uuid4` app/models/ig_draft.py:12.
`news_id`	`UUID`	Foreign key to the `news` table with `CASCADE` delete app/models/ig_draft.py:13.
`hook`	`Text`	The opening line or "hook" for the post app/models/ig_draft.py:14.
`carousel_slides`	`JSONB`	Array of objects containing `{title, body}` for slides app/models/ig_draft.py:15.
`caption`	`Text`	The main body text of the Instagram post app/models/ig_draft.py:16.
`hashtags`	`JSONB`	Array of strings representing post hashtags app/models/ig_draft.py:17.
`cta`	`Text`	Call to Action text app/models/ig_draft.py:18.

Column	Type	Description
`source_line`	`Text`	Mandatory citation of the news source app/models/ig_draft.py:19.
`status`	`Text`	Lifecycle state (Default: `DRAFT`) app/models/ig_draft.py:23.
`variant_of_id`	`UUID`	Self-referencing FK to support multiple versions of one draft app/models/ig_draft.py:25.

1.2 Relationships

The model maintains two primary relationships: * **News**: A `many-to-one` relationship where multiple drafts can belong to a single news item app/models/ig_draft.py:29. * **Variants**: A self-referential relationship (`variant_of`) that allows the system to track iterations or alternative versions of a draft app/models/ig_draft.py:30.

Sources: * app/models/ig_draft.py:9-31

---

2. Status Lifecycle

The `status` column tracks the editorial progress of a draft. Although stored as a `Text` field, it follows a specific state machine logic within the application.

Status Transitions

- **`DRAFT`**: Initial state upon generation.
- **`NEEDS_REVIEW`**: Flagged for manual intervention or refinement.
- **`APPROVED`**: Validated by an editor and ready for publication.
- **`PUBLISHED`**: Final state once the content has been posted to social media.

Sources: * app/models/ig_draft.py:23

---

3. Pydantic Schemas

Data validation and serialization are handled by Pydantic models in `app/schemas/ig_draft.py`. These schemas ensure type safety between the API and the database.

Schema Hierarchy

- **`IGDraftBase`**: Contains common fields shared across all schemas (hook, slides, caption, etc.) app/schemas/ig_draft.py:12-24.
- **`IGDraftCreate`**: Used for initial creation, requiring a `news_id` app/schemas/ig_draft.py:26-28.
- **`IGDraftUpdate`**: All fields are `Optional`, allowing partial updates via `PATCH` requests app/schemas/ig_draft.py:30-41.
- **`IGDraftRead`**: The output schema including database-generated fields like `id`, `created_at`, and `updated_at` app/schemas/ig_draft.py:44-53.

Slide Structure

The `carousel_slides` are validated using the `CarouselSlide` helper class, ensuring every slide has a `title` and a `body` app/schemas/ig_draft.py:7-9.

Sources: * app/schemas/ig_draft.py:7-53

---

4. Data Flow and Entity Mapping

The following diagrams illustrate how the system maps natural language concepts (like a "Social Post") into specific code entities and how the data relates across tables.

Entity Relationship Mapping

This diagram bridges the gap between the logical concept of a social media post and the `IGDraft` ORM entity.

```
graph TD
    subgraph "Natural Language Space"
```

```
        A["Instagram Post"]
        B["Alternative Version"]
        C["Slide Deck"]
    end

    subgraph "Code Entity Space (app/models/ig_draft.py)"
        direction LR
        IGDraft["class IGDraft(Base)"]
        IGDraft -->|"variant_of_id"| IGDraft
        IGDraft -->|"carousel_slides (JSONB)"| Slides["List[dict]"]
    end

    A --- IGDraft
    B --- IGDraft
    C --- Slides
```

Sources: * app/models/ig_draft.py:9-31 * app/schemas/ig_draft.py:14-15

Schema and Model Interaction

This diagram shows how data flows from a creation request through the Pydantic schemas into the SQLAlchemy model.

```
graph LR
    subgraph "Request Handling"
        Req["POST /api/ig/news/{id}/generate"] --> Create["IGDraftCreate"]
    end

    subgraph "Persistence Layer"
        Create -->|"mapped to"| Model["IGDraft (SQLAlchemy)"]
        Model -->|"news_id FK"| NewsTable["news (Table)"]
    end

    subgraph "Response Serialization"
        Model -->|"from_attributes=True"| Read["IGDraftRead"]
    end
```

Sources: * app/schemas/ig_draft.py:26-53 * app/models/ig_draft.py:13

Database Migrations

This section details the database migration infrastructure used by the Althara News Service. The system utilizes **Alembic** to manage schema evolutions for the PostgreSQL database (Neon), with a specific focus on supporting asynchronous operations and automatic URL normalization for `asyncpg`.

Alembic Configuration and Async Engine

The migration system is configured to handle asynchronous database connections, which is a requirement for the `sqlalchemy.ext.asyncio` stack used throughout the application.

Implementation Details

- **`alembic.ini`**: Defines the migration script location as `alembic/` and sets up logging alembic.ini:3-104. It does not store the database URL directly; instead, the URL is injected at runtime via `env.py` alembic.ini:60.
- **`env.py`**: This script bootstraps the migration environment. It imports the `Base.metadata` from the application's database module to enable autogenerate capabilities alembic/env.py:17-21.
- **Async Support**: The `run_async_migrations` function uses `async_engine_from_config` to create a connectable engine and runs the migrations within a synchronous wrapper using `connection.run_sync(do_run_migrations)` alembic/env.py:101-119.

Database URL Normalization

To ensure compatibility with the `asyncpg` driver and the Neon PostgreSQL pooler, the service implements a normalization utility. This logic is shared between the main application and the Alembic environment.

Feature	Logic
Driver Conversion	Replaces `postgresql://` with `postgresql+asyncpg://` alembic/env.py:35-36.
Param Stripping	Removes `sslmode` and `channel_binding` which are incompatible with `asyncpg` alembic/env.py:43-45.
SSL Handling	Relies on `asyncpg` to handle SSL automatically alembic/env.py:30.

Sources: alembic.ini:1-115, alembic/env.py:1-128, app/database.py:12-49

Migration Chain

The database schema has evolved through a series of revisions to support new features like multi-brand support (Althara/Oxono) and Instagram draft management.

Migration Sequence Diagram

The following diagram illustrates the lineage of the database schema from the initial table creation to the latest fields.

"Migration Lineage"

```
graph TD
    subgraph "History"
        A["001_initial"] --> B["7559f42308c6"]
        B --> C["bc2223ec7dd4"]
        C --> D["43db6c86d1d4"]
        D --> E["002"]
    end

    subgraph "Entities Created"
        node_A["news table"]
        node_B["provincia, poblacion"]
        node_C["instagram_post"]
        node_D["althara_content (JSONB)"]
        node_E["ig_drafts table, domain, relevance_score"]
    end

    A -.-> node_A
    B -.-> node_B
    C -.-> node_C
```

```
    D -.-> node_D
    E -.-> node_E
```

Sources: alembic/versions/001_initial_migration_create_news_table.py:1-42, alembic/versions/7559f42308c6_add_provincia_poblacion_fields.py:1-27, alembic/versions/bc2223ec7dd4_add_instagram_post_field.py:1-25, alembic/versions/002_add_domain_relevance_ig_drafts.py:1-62

Detailed Revision History

Revision ID	Name	Key Changes
`001_initial`	Initial Migration	Creates the `news` table with core fields: `id`, `title`, `source`, `url`, `published_at`, `category`, and timestamps alembic/versions/001_initial_migration_create_news_table.py:20-34.
`7559f42308c6`	add_provincia_poblacion_fields	Adds `provincia` and `poblacion` columns to the `news` table for localized real estate filtering alembic/versions/7559f42308c6_add_provincia_poblacion_fields.py:19-21.
`bc2223ec7dd4`	add_instagram_post_field	Adds a `instagram_post` text field to store generated social media copy alembic/versions/bc2223ec7dd4_add_instagram_post_field.py:19-20.
`43db6c86d1d4`	add_althara_content_field	(Implicit in chain) Adds `althara_content` JSONB field for structured data.
`002`	add_domain_relevance_ig_drafts	**Major Update**: Extends `news` with `domain` (real_estate/tech) and `relevance_score`. Creates the `ig_drafts` table with a foreign key to `news.id` alembic/versions/002_add_domain_relevance_ig_drafts.py:18-50.

Data Flow: Code to Database Entities

The following diagram bridges the Python models defined in the application to the physical tables managed by Alembic.

"Model-to-Schema Mapping"

```
classDiagram
    class Base {
        <<DeclarativeBase>>
        +metadata
    }
    class News {
        <<SQLAlchemy Model>>
        +UUID id
        +String title
        +String domain
        +Integer relevance_score
        +JSONB althara_content
    }
    class IGDraft {
        <<SQLAlchemy Model>>
        +UUID id
        +UUID news_id
        +JSONB carousel_slides
        +String status
    }

    Base <|-- News
    Base <|-- IGDraft
    News "1" -- "0..*" IGDraft : ig_drafts relationship

    subgraph "Alembic Migrations"
        M1["001_initial"]
        M2["002_add_domain_relevance"]
    end

    M1 --> News : creates
    M2 --> News : alters (domain/relevance)
    M2 --> IGDraft : creates
```

Sources: alembic/env.py:17-21, alembic/versions/002_add_domain_relevance_ig_drafts.py:25-50, app/database.py:9

Operations

Running Migrations

Migrations are executed using the standard Alembic CLI. The environment must have a valid `DATABASE_URL` configured.

1. **Upgrade to Latest**: `alembic upgrade head` README.md:120
2. **Verify Migration Status**: Check logs for `INFO [alembic.runtime.migration] Running upgrade -> ...` README.md:191-195

Rolling Back Migrations

Rollbacks are handled by the `downgrade` functions defined in each revision script.

- **Revert One Version**: `alembic downgrade -1`
- **Implementation Example**: In revision `7559f42308c6`, the `downgrade` function calls `op.drop_column('news', 'poblacion')` and `op.drop_column('news', 'provincia')` alembic/versions/7559f42308c6_add_provincia_poblacion_fields.py:24-26.

Deployment Integration

On platforms like Render, migrations are typically integrated into the build or pre-deploy command to ensure the schema is updated before the application starts README.md:181-187.

Sources: README.md:105-124, alembic/versions/7559f42308c6_add_provincia_poblacion_fields.py:24-26

Scripts and Automation

The `scripts/` directory contains a collection of standalone Python utilities and shell wrappers designed to manage the news service outside of the standard FastAPI request-response lifecycle. These tools are used for scheduled tasks (via cron), bulk data maintenance, and developer debugging.

System Automation Architecture

The following diagram illustrates how the automation scripts interact with the core application logic and the database.

Automation Logic Flow

```
graph TD
    subgraph "Automation Space"
        CRON["Cron / Scheduler"] --> WRAP["ingest_wrapper.sh"]
        WRAP --> INGEST_PY["ingest_news.py"]
        CLEAN_PY["clean_and_reingest.py"]
        RECAT_PY["recategorize_news.py"]
    end

    subgraph "Code Entity Space"
        INGEST_PY --> RSS_FUNC["ingest_rss_sources()"]
        INGEST_PY --> ADAPT_FUNC["build_all_content()"]
        CLEAN_PY --> DB_DELETE["delete(News)"]
        RECAT_PY --> CAT_KEY["_categorize_by_keywords()"]

        RSS_FUNC --> DB[("AsyncSessionLocal")]
        ADAPT_FUNC --> DB
    end

    subgraph "Data Space"
        DB --> NEWS_TABLE["News Table"]
    end
```

Sources: scripts/ingest_news.py:22-81, scripts/clean_and_reingest.py:22-93, scripts/recategorize_news.py:22-74

8.1 Ingestion and Regeneration Scripts

These scripts facilitate the ingestion of news from external RSS sources and the subsequent transformation of that news into brand-aligned content. They are primarily used to automate the pipeline without manual intervention via the Admin API.

- `ingest_news.py`: The primary entry point for automated ingestion. It calls `ingest_rss_sources` to fetch new items and then iterates through pending records to run `build_all_content`, which generates summaries and Instagram posts scripts/ingest_news.py:22-67.
- `ingest_wrapper.sh`: A shell utility that activates the local virtual environment and executes the Python ingestion script, redirecting output to logs scripts/ingest_wrapper.sh:1-7.
- `recategorize_news.py`: A maintenance utility that re-runs the keyword-based classification logic (`_categorize_by_keywords`) on existing database records to fix historical categorization errors scripts/recategorize_news.py:41-53.
- `clean_and_reingest.py`: A destructive script used in development to wipe the `News` table and restart the ingestion process from scratch scripts/clean_and_reingest.py:34-48.

For details on configuration and execution, see Ingestion and Regeneration Scripts.

Sources: scripts/ingest_news.py:1-92, scripts/ingest_wrapper.sh:1-13, scripts/recategorize_news.py:1-82, scripts/clean_and_reingest.py:1-116

8.2 Maintenance and Cleanup Scripts

These utilities are focused on database hygiene and data auditing. They allow developers to prune irrelevant content or verify the integrity of structured JSON data stored in the database.

- `remove_irrelevant_news.py`: Performs bulk pruning of the database. It typically applies the same guardrails logic used during ingestion to identify and remove news that no longer meets quality standards.
- `clean_all_news.py`: A simple utility to truncate the news table.

- `check_structured_content.py`: Audits the `althara_content` JSONB field across the database to ensure it conforms to the expected schema (hecho, lectura, implicaciones, senales_a_vigilar).

For details on database maintenance, see Maintenance and Cleanup Scripts.

Sources: scripts/clean_and_reingest.py:36-40

8.3 Debugging and Validation Scripts

These scripts are intended for developer use to test specific components of the pipeline in isolation, such as the LLM-based adapters or the RSS filtering logic.

- `test_filter.py`: A unit testing script for the guardrails system, checking if specific titles/summaries are correctly blocked by `DENY_KEYWORDS`.
- `test_api_response.py`: Simulates the serialization logic of the `GET /api/news` endpoint to verify that Pydantic models are correctly handling the database output.
- `debug_ingestion.py`: Provides verbose logging for the RSS fetching process to troubleshoot network or parsing issues with specific feeds.

For details on validation tools, see Debugging and Validation Scripts.

Script-to-Module Mapping

The scripts serve as wrappers around core application logic defined in `app/`.

Script	Core Function Called	Module
`ingest_news.py`	`ingest_rss_sources`	`app.ingestion.rss_ingestor`
`ingest_news.py`	`build_all_content`	`app.adapters.news_adapter`
`recategorize_news.py`	`_categorize_by_keywords`	`app.ingestion.rss_ingestor`
`clean_and_reingest.py`	`build_althara_summary`	`app.adapters.news_adapter`

Sources: scripts/ingest_news.py:14-19, scripts/recategorize_news.py:15-19, scripts/clean_and_reingest.py:14-19

Ingestion and Regeneration Scripts

This section covers the standalone Python and shell scripts located in the `scripts/` directory. These utilities are designed for automated execution (via cron), maintenance tasks, and bulk data transformation outside of the standard FastAPI request-response lifecycle.

Primary Ingestion Pipeline

The primary entry point for automated ingestion is `ingest_news.py`, which coordinates the fetching of raw news and its subsequent transformation into brand-aligned content.

ingest_news.py

This script executes a two-stage pipeline: 1. **Ingestion**: Calls `ingest_rss_sources` to fetch and persist new items from configured RSS feeds scripts/ingest_news.py:26-34. 2. **Adaptation**: Queries the database for `News` records where `althara_summary` or `instagram_post` are null scripts/ingest_news.py:37-41. It then uses `build_all_content` to generate the missing fields scripts/ingest_news.py:46-52.

ingest_wrapper.sh

A shell wrapper used for deployment and cron scheduling. It sets the working directory, activates the virtual environment, and redirects output to a log file scripts/ingest_wrapper.sh:5-7.

Ingestion Logic Flow

The following diagram illustrates the relationship between the ingestion script and the application core:

Ingestion Script to Core Entity Mapping

```
graph TD
    subgraph "Script Space"
        S1["ingest_news.py"]
        S2["ingest_wrapper.sh"]
    end

    subgraph "Code Entity Space"
        F1["ingest_rss_sources()"]
        F2["build_all_content()"]
        M1["News Model"]
        DB[("AsyncSessionLocal")]
    end

    S2 -->|executes| S1
    S1 -->|initializes| DB
    S1 -->|calls| F1
    S1 -->|calls| F2
    F1 -->|persists| M1
    F2 -->|updates| M1
```

Sources: scripts/ingest_news.py:15-18, scripts/ingest_wrapper.sh:7, scripts/ingest_news.py:24-65

Content Regeneration and Migration

These scripts are used when the adaptation logic or the database schema for structured content changes, requiring bulk updates to existing records.

generate_structured_for_all.py

This script targets `News` records where the `althara_content` field is `None` scripts/generate_structured_for_all.py:23-25. It uses `build_all_content_structured` to populate the V2 JSON schema (including `web` and `social` blocks) for pending items scripts/generate_structured_for_all.py:44-51.

regenerate_all_structured.py

Unlike the "pending only" script, this forces an update on **all** records in the database scripts/regenerate_all_structured.py:23-25. It is typically used after a major update to the `news_adapter.py` logic to ensure all historical news items reflect the latest formatting and narrative reconstruction scripts/regenerate_all_structured.py:42-54.

Data Flow for Structured Regeneration

```
sequenceDiagram
    participant Script as regenerate_all_structured.py
    participant DB as AsyncSessionLocal
    participant Adapter as build_all_content_structured()
    participant Model as News ORM

    Script->>DB: select(News)
    DB-->>Script: List[News]
    loop For each News item
        Script->>Adapter: title, raw_summary, category...
        Adapter-->>Script: althara_summary, instagram_post, structured_content
        Script->>Model: update fields
    end
    Script->>DB: session.commit()
```

Sources: scripts/regenerate_all_structured.py:15-54, scripts/generate_structured_for_all.py:15-59

Maintenance and Cleanup

clean_and_reingest.py

A high-impact script used to reset the database state. It performs the following: 1. **Purge**: Executes a `delete(News)` statement to wipe all records scripts/clean_and_reingest.py:36-38. 2. **Re-ingest**: Triggers `ingest_rss_sources` scripts/clean_and_reingest.py:45. 3. **Re-adapt**: Rebuilds summaries using `build_althara_summary` scripts/clean_and_reingest.py:62-67.

recategorize_news.py

Used when the keyword-based classification logic in `rss_ingestor.py` is updated. It iterates through all news items and calls `_categorize_by_keywords` on the existing `title` and `raw_summary` scripts/recategorize_news.py:42-43. If the new category differs from the stored one, the record is updated scripts/recategorize_news.py:45-50.

Summary of Script Capabilities

Script	Target	Primary Function	Core Call
`ingest_news.py`	New/Pending	Daily ingestion and adaptation	`ingest_rss_sources`
`clean_and_reingest.py`	All	Full DB wipe and fresh fetch	`delete(News)`
`recategorize_news.py`	All	Updates `category` field via keywords	`_categorize_by_keywords`
`generate_structured_for_all.py`	Missing `althara_content`	Populates V2 structured JSON	`build_all_content_struct`
`regenerate_all_structured.py`	All	Overwrites all content fields	`build_all_content_struct`

Sources: scripts/ingest_news.py:22-73, scripts/clean_and_reingest.py:22-83, scripts/recategorize_news.py:22-66, scripts/regenerate_all_structured.py:19-75, scripts/generate_structured_for_all.py:19-79

Maintenance and Cleanup Scripts

This section details the standalone utility scripts designed for database maintenance, data integrity auditing, and bulk pruning. These scripts operate directly on the database using `AsyncSessionLocal` and are intended for administrative use via CLI or automated maintenance windows.

Overview of Maintenance Utilities

The maintenance suite provides tools for four primary operational tasks: 1. **Relevance Pruning**: Removing historical data that no longer meets the system's topical guardrails. 2. **Full Reset**: Completely wiping the news database for re-indexing. 3. **Content Auditing**: Verifying the state of JSONB structured content fields across the dataset. 4. **Content Inspection**: Deep-diving into the transformed data for specific records.

Maintenance Data Flow

The following diagram illustrates how maintenance scripts interact with the `News` model and the underlying PostgreSQL storage.

Maintenance Script Interaction Map

```
graph TD
    subgraph "CLI Scripts"
        RM["remove_irrelevant_news.py"]
        CLN["clean_all_news.py"]
        CHK["check_structured_content.py"]
        VW["view_structured_content.py"]
    end

    subgraph "Code Entities"
        ASL["AsyncSessionLocal"]
        NM["News Model"]
        GR["_is_relevant_to_real_estate"]
    end

    subgraph "Database Space"
```

```
        DB[("PostgreSQL Table: news")]
    end

    RM -->|"Uses"| GR
    RM -->|"Queries/Deletes"| ASL
    CLN -->|"Executes DELETE"| ASL
    CHK -->|"Counts JSONB"| ASL
    VW -->|"Selects ID"| ASL

    ASL -->|"ORMs"| NM
    NM <-->|"SQL"| DB
```

Sources: scripts/remove_irrelevant_news.py:17-21, scripts/clean_all_news.py:12-15, scripts/check_structured_content.py:13-16, scripts/view_structured_content.py:14-17

Irrelevant News Removal

The `remove_irrelevant_news.py` script identifies and removes news items that fail the real estate relevance check. This is particularly useful after updating the `DENY_KEYWORDS` or `ALLOW_KEYWORDS` in the system constants to purge legacy "noise" from the database.

Implementation Details

The script uses the internal `_is_relevant_to_real_estate` logic from the `rss_ingestor` to ensure consistency between ingestion and maintenance scripts/remove_irrelevant_news.py:20-20.

- **Dry Run Mode**: Activated via `--dry-run` or `-d`, it performs the analysis and prints a detailed report without executing the `delete` statement scripts/remove_irrelevant_news.py:24-33.
- **Grouping**: It aggregates candidates for deletion by `source` and `category` to help administrators identify problematic RSS feeds scripts/remove_irrelevant_news.py:73-80.
- **Bulk Deletion**: Uses `delete(News).where(News.id.in_(ids_to_delete))` for efficient batch removal scripts/remove_irrelevant_news.py:118-120.

Removal Logic Flow

```
flowchart TD
    START["Start Script"] --> FETCH["Select all News from DB"]
    FETCH --> LOOP["Iterate through News items"]
    LOOP --> CHECK{"passes_guardrails?"}
    CHECK -- "No" --> LIST["Add to irrelevant_news list"]
    CHECK -- "Yes" --> KEEP["Add to relevant_news list"]
    LIST --> REPORT["Generate Detail Report (by source/category)"]
    REPORT --> MODE{"Is Dry Run?"}
    MODE -- "Yes" --> END["Exit without changes"]
    MODE -- "No" --> DEL["Execute Bulk DELETE by IDs"]
    DEL --> COMMIT["session.commit()"]
    COMMIT --> END
```

Sources: scripts/remove_irrelevant_news.py:24-139

Database Full Wipe

The `clean_all_news.py` script provides a nuclear option to empty the `news` table. This is typically used in staging environments or when a full re-ingestion with new logic is required.

- **Safety**: It prints a prominent warning and the total count of news items before proceeding scripts/clean_all_news.py:30-36.
- **Atomic Operation**: It executes a single `delete(News)` statement and commits the transaction scripts/clean_all_news.py:39-41.

Sources: scripts/clean_all_news.py:18-52

Structured Content Auditing

With the introduction of the `althara_content` JSONB field (Revision `43db6c86d1d4`), it became necessary to track which records have been processed by the News Adapter.

Audit Script: `check_structured_content.py`

This script provides a statistical overview of the database's adaptation state: * **Metrics**: Calculates total news vs. news where `althara_content` is not null scripts/check_structured_content.py:23-32. * **Schema Validation**: It inspects the first available structured record and prints the presence of `web` and `instagram` sub-keys to verify they conform to the expected v2.0 schema scripts/check_structured_content.py:61-76.

Inspection Script: `view_structured_content.py`

A developer tool for deep-inspecting the JSON transformation of a specific article. * **Lookup**: Supports finding news by UUID or by its index in the database scripts/view_structured_content.py:23-35. * **Pretty Print**: Parses the `althara_content` JSONB and formats the `web`, `instagram`, and `qa` sections for terminal readability scripts/view_structured_content.py:63-113.

Structured Content Data Structure (althara_content) | Field | Source Logic | Purpose | | :--- | :--- | :--- | | `web` | `build_structured_content` | Title, Deck, Hecho, Lectura, Implicaciones, Señales. | | `instagram` | `build_instagram_post` | Hook, Slides, Caption, CTA. | | `qa` | Adapter Validation | `facts_used` and `unknown_or_missing` lists. | | `metadata` | Adapter | Generation timestamp, category, and random seed. |

Sources: scripts/check_structured_content.py:19-102, scripts/view_structured_content.py:20-136, alembic/versions/43db6c86d1d4_add_althara_content_field.py:20-21

Summary of Maintenance Commands

Script	Command	Purpose
Pruning	`python3 scripts/remove_irrelevant_news.py --dry-run`	Preview news to be deleted based on guardrails.
Wipe	`python3 scripts/clean_all_news.py`	Delete all news records from the database.

Script	Command	Purpose
Audit	`python3 scripts/check_structured_content.py`	Show stats on adapted vs. raw news items.
Inspect	`python3 scripts/view_structured_content.py <UUID>`	View formatted JSONB content for a specific news ID.

Sources: scripts/remove_irrelevant_news.py:142-143, scripts/view_structured_content.py:139-140

Debugging and Validation Scripts

This page details the developer-focused scripts used to validate system behavior, test content generation logic, and debug the ingestion pipeline. These scripts operate independently of the FastAPI application but utilize the same core logic and database connections to ensure parity between development tests and production behavior.

Ingestion and Guardrails Debugging

These scripts focus on the entry point of the news pipeline, specifically the filtering logic that determines if a news item is relevant to the Real Estate domain.

`debug_ingestion.py`

This script provides a verbose walkthrough of the RSS ingestion process. It connects to the database, counts existing records, and then iterates through the first three sources in `RSS_SOURCES` app/ingestion/rss_ingestor.py:15-15. For each source, it: 1. Fetches the raw feed using `httpx`. 2. Checks for URL existence in the `News` table to prevent duplicates scripts/debug_ingestion.py:72-74. 3. Runs the first-pass relevance filter `_is_relevant_to_real_estate` app/ingestion/rss_ingestor.py:9-9 and reports the specific reason for acceptance or rejection.

Data Flow: Ingestion Debugging

```
graph TD
    subgraph "Code Entity Space"
        S1["debug_ingestion.py"]
        F1["_is_relevant_to_real_estate()"]
        M1["News Model"]
        D1["AsyncSessionLocal"]
    end

    subgraph "Natural Language Space"
        RSS["External RSS Feeds"]
        RE["Relevance Decision"]
        DB["PostgreSQL Storage"]
```

```
    end

    S1 -->|fetches| RSS
    S1 -->|calls| F1
    F1 -->|returns| RE
    S1 -->|queries| D1
    D1 -->|accesses| M1
    M1 -->|reflects| DB
```

Sources: scripts/debug_ingestion.py:21-133, app/ingestion/rss_ingestor.py:9-15

`test_filter.py` and `test_filter_with_summary.py`

These are lightweight unit tests for the guardrails logic. * `test_filter.py` : Validates the `_is_relevant_to_real_estate` function against a hardcoded list of titles like "Giro inesperado en la okupación" scripts/test_filter.py:11-19. * `test_filter_with_summary.py` : Extends this by passing both a title and a summary string to the filter to simulate the more complex matching that occurs when RSS feeds provide descriptive snippets scripts/test_filter_with_summary.py:12-25.

Sources: scripts/test_filter.py:1-25, scripts/test_filter_with_summary.py:1-35

Content Adaptation Validation

These scripts validate the transformation of raw news into structured formats for the web and social media.

`test_instagram_posts.py`

This script tests the generation of Instagram captions and saves them to the database. It retrieves up to 5 news items and passes them through `build_all_content` app/adapters/news_adapter.py:16-16.

- **Logic**: It populates the `instagram_post` column and, if missing, the `althara_summary` column for the selected news records scripts/test_instagram_posts.py:68-72.
- **Output**: Prints the generated text and character length to the console for manual review of the brand voice.

`test_structured_content.py`

Validates the v2.0 JSON schema generation used for the "News Studio" UI and the mobile-optimized web view. * **Functionality**: Calls `build_all_content_structured` app/adapters/news_adapter.py:15-15 and inspects the resulting dictionary. * **Validation**: It breaks down the output into `web`, `instagram`, and `qa` blocks, verifying fields like `hecho`, `lectura`, and `senales_a_vigilar` scripts/test_structured_content.py:51-72. * **Persistence**: Saves the resulting JSON into the `althara_content` JSONB column scripts/test_structured_content.py:75-76.

Entity Mapping: Structured Content Generation

```
graph LR
    subgraph "Code Entity Space"
        T1["test_structured_content.py"]
        A1["build_all_content_structured()"]
        N1["News.althara_content (JSONB)"]
    end

    subgraph "Data Structures"
        W1["web: {hecho, lectura, implicaciones}"]
        I1["instagram: {hook, carousel_slides, caption}"]
        Q1["qa: {facts_used, unknown_or_missing}"]
    end

    T1 --> A1
    A1 --> W1
    A1 --> I1
    A1 --> Q1
    W1 & I1 & Q1 --> N1
```

Sources: scripts/test_structured_content.py:18-84, app/adapters/news_adapter.py:15-16

System and API Verification

`check_instagram_posts.py`

A diagnostic utility to audit the state of the database. It provides a summary of: * Total news items in the database scripts/check_instagram_posts.py:22-24. * Percentage of news items that have an `instagram_post` generated scripts/check_instagram_posts.py:27-29. * A list of IDs for news items that are missing social media content, facilitating targeted regeneration scripts/check_instagram_posts.py:54-62.

`test_api_response.py`

This script simulates the internal behavior of the FastAPI `GET /api/news` endpoint without requiring the server to be running. It is critical for ensuring that SQLAlchemy models are correctly serialized into Pydantic schemas.

- **Implementation**: It manually executes a query on the `News` table, orders by `published_at.desc()`, and applies a limit scripts/test_api_response.py:25-32.
- **Serialization Test**: It passes the ORM objects through `NewsRead.model_validate(news)` and wraps them in a `PaginatedResponse` scripts/test_api_response.py:40-49.
- **Verification**: Specifically checks if the `instagram_post` field survives the JSON serialization process, which helps debug issues where fields might be excluded by schema configurations scripts/test_api_response.py:64-65.

Sources: scripts/test_api_response.py:21-89, scripts/check_instagram_posts.py:18-104, app/schemas/news.py:17-17

Summary of Scripts

Script	Primary Function	Key Code Reference
`test_filter.py`	Unit test for RE relevance	`_is_relevant_to_real_estate`
`debug_ingestion.py`		`ingest_rss_sources`

Script	Primary Function	Key Code Reference
	Trace RSS fetch & filter	
`test_instagram_posts.py`	Generate/Save IG captions	`build_all_content`
`test_structured_content.py`	Validate v2.0 JSON schema	`althara_content` (JSONB)
`test_api_response.py`	Mock FastAPI serialization	`NewsRead` & `PaginatedResponse`
`check_instagram_posts.py`	Database content audit	`News.instagram_post.is_(None)`

Sources: scripts/test_filter.py:9-9, scripts/debug_ingestion.py:15-15, scripts/test_instagram_posts.py:16-16, scripts/test_structured_content.py:15-15, scripts/test_api_response.py:17-17, scripts/check_instagram_posts.py:54-54

Testing

This section provides an overview of the test suite for the Althara News Service. The testing architecture is built on `pytest` and utilizes asynchronous fixtures to validate the ingestion, classification, adaptation, and API layers.

The test suite is located in the `tests/` directory and is configured to run in asynchronous mode by default.

Test Configuration

The project uses `pytest-asyncio` to handle the asynchronous nature of the FastAPI application and its database interactions. The configuration in `pytest.ini` ensures that the event loop scope is maintained across sessions for efficiency.

Configuration Key	Value	Description
`asyncio_mode`	`auto`	Automatically detects async test functions.
`asyncio_default_test_loop_scope`	`session`	Reuses the event loop across the test session.
`testpaths`	`tests`	Defines the root directory for test discovery.

Sources: - pytest.ini:1-5

Testing Architecture

The testing suite bridges the gap between raw input data (RSS feeds/scraped HTML) and the final structured outputs (JSONB content/IG Drafts). The following diagram illustrates how the test fixtures and modules interact with the core system entities.

Test Suite to Code Entity Mapping

```
graph TD
    subgraph "Natural Language Space (Fixtures)"
```

```
        F1["sample_tech_news"]
        F2["sample_real_estate_news"]
    end

    subgraph "Code Entity Space (System Under Test)"
        C1["app.routers.news"]
        C2["app.adapters.ig_adapter"]
        C3["app.classifiers.tech_classifier"]
        C4["app.utils.text_compaction"]
    end

    subgraph "Test Modules"
        T1["test_api.py"]
        T2["test_ig_adapter.py"]
        T3["test_tech_classifier.py"]
        T4["test_text_compaction.py"]
    end

    F1 --> T2
    F1 --> T3
    F2 --> T1

    T1 -.-> C1
    T2 -.-> C2
    T3 -.-> C3
    T4 -.-> C4
```

Sources: - tests/conftest.py:7-36 - tests/pytest.ini:1-5

Shared Fixtures

The `conftest.py` file provides reusable mock objects that simulate the `News` ORM model. These fixtures allow tests to verify logic without requiring a live database connection in every instance.

- **`sample_tech_news`**: Simulates a news item for the Oxono brand, including `domain='tech'` and `relevance_score` tests/conftest.py:7-20.
- **`sample_real_estate_news`**: Simulates a news item for the Althara brand, including `domain='real_estate'` and `althara_summary` tests/conftest.py:23-36.

Sources: - tests/conftest.py:7-36

Test Coverage Overview

The test suite is divided into specialized modules targeting different layers of the application:

1. **API Integration**: Validates FastAPI endpoints, request validation, and response serialization.
2. **Adapter Logic**: Ensures the `ig_adapter.py` correctly transforms `News` records into `IGDraft` objects with the appropriate slides and captions.
3. **Classification & Scoring**: Tests the `tech_classifier.py` for its ability to assign categories (e.g., `AI_ML`) and calculate relevance based on keyword density.
4. **Utility Validation**: Unit tests for text processing, such as sentence splitting and HTML stripping.

System Component Testing Flow

```
sequenceDiagram
    participant T as Test Runner (pytest)
    participant F as Fixtures (conftest.py)
    participant S as System Logic (Adapters/Classifiers)

    T->>F: Request sample_tech_news
    F-->>T: Return Mock News Object
    T->>S: Pass Mock to ig_adapter.generate_drafts()
    S-->>T: Return IGDraft object
    T->>T: Assert status == 'DRAFT'
```

Sources: - tests/conftest.py:1-37

Child Pages

For detailed documentation on specific test modules and implementation details, see:

- **Test Suite Reference**: Detailed breakdown of `test_api.py`, `test_ig_adapter.py`, `test_tech_classifier.py`, `test_text_compaction.py`, and `test_utils.py`.

Test Suite Reference

The Althara News Service utilizes a comprehensive test suite powered by `pytest` to ensure the reliability of the news ingestion pipeline, content transformation logic, and API integrity. The suite covers unit tests for text utilities, domain-specific classification logic, and integration tests for the FastAPI endpoints.

Core Configuration and Fixtures

The testing environment is configured via `tests/conftest.py` tests/conftest.py:1-5, which provides shared fixtures representing the two primary domains: `tech` (Oxono) and `real_estate` (Althara). These fixtures simulate the `News` ORM model structure to test adapters and classifiers without requiring a live database connection.

Available Fixtures

- `sample_tech_news`: Simulates a tech-domain news object with fields like `AI_ML` category and a relevance score tests/conftest.py:8-20.
- `sample_real_estate_news`: Simulates a real estate news object with `althara_summary` and domain-specific categories tests/conftest.py:24-36.

Sources: tests/conftest.py:1-37

API Integration Tests (`test_api.py`)

The `test_api.py` module performs integration testing on the FastAPI application using an `AsyncClient` with `ASGITransport` tests/test_api.py:3-15. This ensures that tests run within the same event loop as the application.

Key Test Scenarios

- **Health Check**: Validates the `/api/health` endpoint returns a `200 OK` status tests/test_api.py:18-21.
- **Domain Filtering**: Verifies that the `GET /api/news` endpoint correctly respects the `domain` query parameter for both `tech` and `real_estate` tests/test_api.py:24-38.

Sources: tests/test_api.py:1-38

Instagram Adapter Tests (`test_ig_adapter.py`)

This module validates the logic in `ig_adapter.py` for generating Instagram drafts. It ensures that the transformation from a `News` object to an `IGDraft` schema adheres to brand-specific constraints.

Logic Validation

- **Slide Composition**: Asserts that exactly `SLIDES_COUNT` (3) slides are generated, each containing a title and a body within `BODY_MAX` characters tests/test_ig_adapter.py:14-19.
- **Oxono (Tech) Specifics**: Validates that tech drafts include the correct hashtag count (between `OXONO_HASHTAGS_MIN` and `OXONO_HASHTAGS_MAX`) and a sufficiently long caption tests/test_ig_adapter.py:14-23.
- **Variant Generation**: Ensures that the `generate_variants` function produces the requested number of unique draft objects tests/test_ig_adapter.py:34-39.

Data Flow: News to IG Draft

The following diagram illustrates how the test suite verifies the transformation from a `News` entity to an `IGDraft` entity.

Diagram: IG Adapter Entity Transformation

```
graph TD
    subgraph "Natural Language Space"
        A["Raw News Article"] -- "Ingested" --> B["News Record"]
```

```
    end

    subgraph "Code Entity Space"
        B -- "sample_tech_news (Fixture)" --> C["generate_ig_draft()"]
        C -- "Produces" --> D["IGDraft Dictionary"]

        subgraph "Assertions in test_ig_adapter.py"
            D1["len(carousel_slides) == 3"]
            D2["len(hashtags) within Oxono limits"]
            D3["caption length >= 100"]
        end

        D --> D1
        D --> D2
        D --> D3
    end
```

Sources: tests/test_ig_adapter.py:1-39, app/adapters/ig_adapter.py:3-10

Tech Classifier Tests (`test_tech_classifier.py`)

These tests focus on the `Oxono` domain's automated categorization and scoring logic defined in `app/tech/classifier.py`.

Functional Coverage

- **Category Inference**: Validates that `infer_category` maps titles to `TechNewsCategory` enums (e.g., "GPT-5" → `AI_ML`, "Beta" → `RELEASE_UPDATE`, "Startup" → `STARTUPS`) tests/test_tech_classifier.py:7-22.
- **Tag Extraction**: Ensures `extract_tags` identifies relevant keywords like "ai" or "ia" from titles tests/test_tech_classifier.py:24-27.
- **Relevance Scoring**: Checks that `compute_relevance_score` produces values within the 0-100 range, prioritizing high-value categories like `AI_ML` over `OTHER_TECH` tests/test_tech_classifier.py:29-37.

Sources: tests/test_tech_classifier.py:1-37, app/tech/classifier.py:3-4

Text Utilities and Compaction

Testing for shared utilities is split between `test_utils.py` and `test_text_compaction.py`.

HTML and RSS Utilities (`test_utils.py`)

- **`strip_html_tags`**: Tests the removal of tags, unescaping of entities, and repair of "mojibake" (encoding errors) using `ftfy` tests/test_utils.py:11-32.
- **`parse_published_date`**: Verifies that dates are correctly extracted from RSS entry objects and converted to UTC `datetime` objects tests/test_utils.py:41-59.
- **`passes_guardrails`**: Tests the keyword-based filtering logic, including `deny` lists and `strict_allow` requirements tests/test_utils.py:61-78.

Compaction and Truncation (`test_text_compaction.py`)

- **`extract_key_sentences`**: Asserts that only complete sentences (ending in `.!?`) that fit within `max_chars` are returned tests/test_text_compaction.py:12-21.
- **`truncate_at_sentence`**: Ensures text is never cut mid-word and avoids leaving dangling prepositions (e.g., ending a sentence with "a") tests/test_text_compaction.py:23-38.
- **`compose_caption_blocks`**: Validates the assembly of the final Instagram caption, ensuring it includes the source line and adheres to total length limits tests/test_text_compaction.py:59-69.

Diagram: Text Utility Logic Flow

```
graph LR
    subgraph "Input Processing"
        RawHTML["Raw HTML/RSS Content"] -- "strip_html_tags()" --> CleanText["Cleaned String"]
    end

    subgraph "Sentence Management"
        CleanText -- "split_sentences()" --> SentList["List of Sentences"]
        SentList -- "truncate_at_sentence()" --> ShortText["Truncated Content"]
    end

    subgraph "Validation in test_text_compaction.py"
        ShortText -- "Assert" --> V1["No mid-word cuts"]
```

```
        ShortText -- "Assert" --> V2["No dangling prepositions"]
    end
```

Sources: tests/test_utils.py:1-78, tests/test_text_compaction.py:1-69, app/utils/html_utils.py:18-40

Deployment and Infrastructure

This section describes the deployment architecture and infrastructure configuration for the Althara News Service. The service is designed to run as a containerized web application, primarily targeting the **Render.com** platform, utilizing a **Python 3.11.9** runtime and a **PostgreSQL** database.

Infrastructure Overview

The infrastructure follows a standard cloud-native pattern where the application server is decoupled from the persistence layer. The service requires a high-performance asynchronous environment to handle concurrent RSS ingestion and AI-driven content transformation.

Runtime and Build Process

The service specifies a strict Python version to ensure compatibility across environments runtime.txt:1-1. The build process involves installing dependencies from `requirements.txt` and automatically applying database schema changes.

Core Service Configuration

The application is configured via Pydantic Settings, which validates required environment variables at startup app/config.py:10-13. Key configurations include: * **Database Connectivity**: Managed via `DATABASE_URL` app/config.py:13-13. * **Authentication**: Secured via `INGEST_TOKEN` for API automation and `UI_USER` / `UI_PASS` for the News Studio app/config.py:23-25. * **Pipeline Behavior**: Limits for ingestion and flags for automatic Instagram draft generation app/config.py:28-32.

System Architecture Diagram

The following diagram illustrates the relationship between the deployment configuration, the application entry point, and the external infrastructure.

Deployment Component Mapping

```
graph TD
    subgraph "Render.com Cloud"
        A["render.yaml"] -- "configures" --> B["Web Service"]
        B -- "executes" --> C["uvicorn app.main:app"]
        B -- "runs build" --> D["pip install + alembic upgrade"]
    end

    subgraph "Environment Variables (Settings Class)"
        E["DATABASE_URL"]
        F["INGEST_TOKEN"]
        G["UI_USER / UI_PASS"]
    end

    subgraph "Persistence"
        H[("Neon PostgreSQL")]
    end

    B -.-> E
    B -.-> F
    B -.-> G
    C -- "Async Connection" --> H
```

Sources: render.yaml:1-10, app/config.py:10-45, runtime.txt:1-1

Deployment Lifecycle

The deployment lifecycle is automated through the `render.yaml` blueprint. It ensures that every code push results in a consistent environment where the database schema is always in sync with the application code.

Build and Start Sequence

1. **Environment Setup**: Render identifies the Python version from `runtime.txt` runtime.txt:1-1.
2. **Dependency Installation**: The `buildCommand` upgrades `pip` and installs the required packages render.yaml:5-5.
3. **Database Migration**: As part of the build step, `alembic upgrade head` is executed to ensure the PostgreSQL schema matches the ORM models render.yaml:5-5.
4. **Service Execution**: The application starts using `uvicorn`, binding to the port provided by the environment render.yaml:6-6.

For a detailed breakdown of the Render configuration and database hosting, see Render Deployment.

Build Command Mapping

```
graph LR
    subgraph "build.sh / render.yaml"
        direction TB
        STEP1["'pip install'"]
        STEP2["'alembic upgrade head'"]
    end

    subgraph "Code Entities"
        direction TB
        REQ["requirements.txt"]
        ALB["alembic/env.py"]
    end

    STEP1 --> REQ
    STEP2 --> ALB
```

Sources: render.yaml:5-6, build.sh:1-16

Automation and Operations

Beyond the initial deployment, the service relies on scheduled tasks and protected endpoints to maintain the news pipeline.

- **Ingestion Scheduling**: Since the service does not include an internal cron engine, ingestion is triggered via external HTTP calls to the Admin API.
- **Security**: Sensitive operations, such as triggering the ingestion pipeline or accessing the News Studio, are protected by tokens and basic authentication app/config.py:23-25.
- **Scalability**: The service uses asynchronous drivers (`asyncpg`) to handle I/O-bound tasks like fetching multiple RSS feeds simultaneously.

For details on how to set up cron jobs and secure the pipeline, see Automation and Scheduling.

Child Pages

- Render Deployment: Service definitions, `render.yaml` specifics, and Neon PostgreSQL integration.
- Automation and Scheduling: Cron job configuration, `INGEST_TOKEN` authentication, and rate limiting.

Sources: app/config.py:1-45, render.yaml:1-10, build.sh:1-16, runtime.txt:1-1

Render Deployment

This page details the infrastructure configuration and deployment lifecycle for the Althara News Service on the **Render** platform. The service is architected as a Python web service utilizing a managed **Neon PostgreSQL** database, with automated schema management via Alembic.

Infrastructure Configuration

The deployment is managed through a `render.yaml` Blueprint specification. This file defines the service type, environment, and the essential commands required to build and execute the application.

Service Definition

The service is defined as a `web` type, which Render uses to expose the FastAPI application to the internet render.yaml:1-4.

Parameter	Value	Description
Type	`web`	Public-facing web service.
Name	`althara-news-service`	The identifier in the Render dashboard.
Environment	`python`	Native Python runtime environment.
Build Command	`pip install... && alembic upgrade head`	Prepares the environment and migrates the DB.
Start Command	`uvicorn app.main:app...`	Launches the FastAPI ASGI server.

Environment Variables

The primary requirement for the service is the `DATABASE_URL` render.yaml:8-9. This variable is typically provided by a Neon PostgreSQL instance. Because the application uses `asyncpg` for asynchronous database operations, the URL is automatically normalized by the application logic app/database.py:12-49 and the migration engine alembic/env.py:24-61.

Sources: render.yaml:1-10, app/database.py:52-57, alembic/env.py:64-68

Build and Deployment Lifecycle

The deployment process follows a specific sequence to ensure the database schema is synchronized with the code before the web server starts handling requests.

1. Build Phase

The `buildCommand` executes two critical tasks: 1. **Dependency Installation**: It upgrades `pip` and installs requirements from `requirements.txt` render.yaml:5. 2. **Schema Migration**: It runs `alembic upgrade head` render.yaml:5. This ensures that any new tables or column changes (such as the `althara_content` or `domain` fields) are applied to the Neon database before the new code version goes live.

A `build.sh` script is also available to handle more complex environments, ensuring only precompiled binary wheels are used to speed up the process build.sh:1-14.

2. Runtime Phase

The `startCommand` initializes the server using `uvicorn` render.yaml:6. * **Host**: `0.0.0.0` to allow external traffic. * **Port**: Dynamically assigned by Render via the `$PORT` environment variable. * **App Entrypoint**: `app.main:app`, pointing to the FastAPI instance.

Deployment Flow Diagram

The following diagram illustrates the relationship between the Render environment, the build commands, and the database synchronization.

Render Deployment Architecture

```
graph TD
    subgraph "Render Platform"
        [render.yaml] --> |"Defines"| WS["Web Service: althara-news-service"]
        WS --> |"1. buildCommand"| BC["pip install & alembic upgrade head"]
        WS --> |"2. startCommand"| SC["uvicorn app.main:app"]
    end

    subgraph "External Infrastructure"
        BC --> |"Apply Migrations"| DB[("Neon PostgreSQL")]
        SC --> |"Connect (asyncpg)"| DB
    end

    subgraph "Code Entity Space"
        BC -.-> |"Executes"| AL["alembic/env.py"]
        SC -.-> |"Loads"| MA["app/main:app"]
        MA -.-> |"Uses"| ADB["app/database.py"]
    end
```

Sources: render.yaml:1-6, build.sh:1-16, alembic/env.py:101-121, app/database.py:68-81

Automated Database Migrations

A key feature of the Render deployment is the automatic execution of Alembic migrations. This prevents "out-of-sync" errors where the application code expects a schema that the database does not yet have.

Database URL Normalization

Since Render and Neon often provide standard `postgresql://` URLs, the service includes a normalization utility to make them compatible with `sqlalchemy.ext.asyncio`.

1. **Protocol Conversion**: Changes `postgresql://` to `postgresql+asyncpg://` app/database.py:35-36.

2. **Parameter Stripping**: Removes `sslmode` and `channel_binding` parameters because `asyncpg` handles SSL automatically and may fail if these standard Postgres parameters are present app/database.py:31-33.

Async Migration Execution

The `alembic/env.py` script is configured to run migrations asynchronously to match the application's database driver.

- **`get_url()`**: Retrieves and normalizes the `DATABASE_URL` alembic/env.py:64-68.
- **`run_async_migrations()`**: Creates an `async_engine_from_config` and runs the migration context within an async connection alembic/env.py:101-121.

Database Connection Flow

```
graph LR
    subgraph "Environment Space"
        EV["DATABASE_URL (Standard)"]
    end

    subgraph "Normalization Logic"
        direction TB
        N1["normalize_database_url()"]
        N2["Replace: postgresql+asyncpg://"]
        N3["Remove: sslmode"]
    end

    subgraph "Database Clients"
        AL["Alembic (alembic/env.py)"]
        APP["FastAPI (app/database.py)"]
    end

    EV --> N1
    N1 --> N2
    N2 --> N3
    N3 --> AL
    N3 --> APP
    AL --> |"alembic upgrade head"| NEON[("Neon DB")]
    APP --> |"AsyncSession"| NEON
```

Sources: app/database.py:12-49, alembic/env.py:24-61, alembic/env.py:101-121

Summary of Deployment Requirements

- **Runtime**: Python 3.11.9 (as specified in `build.sh` context) build.sh:3.
- **Database**: PostgreSQL (Neon recommended).
- **Key Dependencies**: `uvicorn`, `fastapi`, `sqlalchemy`, `alembic`, `asyncpg`.
- **Secrets**: `DATABASE_URL` must be set in the Render dashboard render.yaml:8-9.

Automation and Scheduling

This page details the mechanisms for automating the news ingestion and adaptation pipeline. The system supports both remote scheduling via HTTP triggers and local scheduling via shell scripts, protected by authentication and rate-limiting layers.

Scheduling Options

The ingestion pipeline, which fetches news from RSS sources and adapts them into brand-specific content, can be triggered through two primary methods:

1. Remote Scheduling (cron-job.org)

The most common production method is using an external service like `cron-job.org` to call the service's Admin API. The recommended endpoint for a full pipeline execution is `POST /api/admin/ingest-and-adapt`. This endpoint triggers the sequence of fetching raw news, filtering through guardrails, and generating structured content for both web and Instagram scripts/check_structured_content.py:97-100.

2. Local Scheduling (Local Cron)

For environments where the service is accessible via the local filesystem, a shell wrapper is provided to execute the pipeline as a background process.

- **`ingest_wrapper.sh`**: A shell script that sets up the environment by navigating to the project directory, activating the Python virtual environment, and executing the ingestion script scripts/ingest_wrapper.sh:5-7.
- **`ingest_news.py`**: The underlying Python script that initializes a database session, calls `ingest_rss_sources`, and then iterates through pending news to run `build_all_content` scripts/ingest_news.py:24-52.

Pipeline Data Flow (Automated)

The following diagram illustrates the flow of data when the ingestion script is triggered automatically.

Automated Ingestion Logic

```
graph TD
    subgraph "Execution Layer"
        Cron["Cron Job / Wrapper"] -- "Executes" --> Script["ingest_news.py"]
    end

    subgraph "Core Logic (scripts/ingest_news.py)"
        Script -- "1. ingest_rss_sources()" --> RSS["RSS Ingestor"]
        RSS -- "Persists News" --> DB[("AsyncSessionLocal")]
        Script -- "2. select(News).where(pending)" --> Query["Pending News Query"]
        Query -- "3. build_all_content()" --> Adapter["news_adapter.py"]
        Adapter -- "4. Update Fields" --> DB
    end

    DB -- "althara_summary" --> NewsEntity["News Record"]
    DB -- "instagram_post" --> NewsEntity
```

Sources: scripts/ingest_news.py:22-67, scripts/ingest_wrapper.sh:1-7

Authentication and Security

Protected endpoints, specifically those under `/api/admin` and `/api/ig`, require authentication to prevent unauthorized pipeline triggers.

INGEST_TOKEN Header

The system uses a simple token-based authentication for automation. The `INGEST_TOKEN` is defined in the environment variables and loaded via the `Settings` class app/config.py:25-25. Requests to protected endpoints must include this token in the headers to bypass security gates.

Rate Limiting

To ensure system stability and prevent abuse, the `RateLimitMiddleware` applies different constraints based on the endpoint type. It uses an in-memory `RateLimiter` that tracks requests per IP address using a sliding window app/middleware.py:15-24.

Endpoint Type	Limit	Implementation
Public (`/api/news` , etc.)	100 req/min	`public_rate_limiter` app/middleware.py:64-64
Admin (`/api/admin/` , `/api/tech/admin/`)	10 req/min	`admin_rate_limiter` app/middleware.py:65-65
Health (`/api/health`)	Unlimited	Skipped in `dispatch` app/middleware.py:94-95

Rate Limiting Sequence

```
sequenceDiagram
    participant Client
    participant Middleware as "RateLimitMiddleware"
    participant Limiter as "RateLimiter"
    participant App as "FastAPI Router"

    Client->>Middleware: Request to /api/admin/ingest
    Middleware->>Limiter: is_allowed(client_ip)
    Note over Limiter: Sliding window check (60s)

    alt Limit Exceeded
        Limiter-->>Middleware: (False, 0)
        Middleware-->>Client: 429 Too Many Requests (Retry-After header)
    else Limit OK
        Limiter-->>Middleware: (True, remaining)
        Middleware->>App: call_next(request)
        App-->>Middleware: Response
        Middleware-->>Client: Response + X-RateLimit headers
    end
```

Sources: app/middleware.py:87-131, app/middleware.py:26-49

Automation Configuration

Behavior during automated runs is governed by several flags in `app/config.py` :

- **`REAL_ESTATE_RSS_LIMIT_PER_SOURCE`** : Controls the maximum number of items fetched per source during a single ingestion run (default: 10) app/config.py:29-29.

- `AUTO_GENERATE_IG_AFTER_INGEST`: When set to `True`, the pipeline automatically triggers the generation of Instagram drafts immediately after news ingestion app/config.py:32-32.
- `DATABASE_URL`: Essential for the scripts to connect to the database (e.g., Neon PostgreSQL in production) app/config.py:13-13.

Maintenance and Auditing

The `scripts/` directory contains utilities to audit the results of automated tasks:

- `check_structured_content.py`: Provides a summary of how many news items have successfully been processed into the `althara_content` JSONB field scripts/check_structured_content.py:19-43.
- **Logging**: The `ingest_wrapper.sh` redirects all output to `/tmp/althara_ingest.log`, allowing developers to review the history of cron executions scripts/ingest_wrapper.sh:7-7.

Sources: app/config.py:10-45, scripts/check_structured_content.py:1-106, scripts/ingest_wrapper.sh:1-7

Glossary

This page provides definitions for codebase-specific terms, domain concepts, and architectural components used in the Althara News Service. It serves as a reference for onboarding engineers to understand the nomenclature and data structures used across the ingestion and transformation pipelines.

Brand and Domain Concepts

The system is multi-tenant by design, supporting two distinct brands with unique voices and topical domains.

Term	Definition	Implementation Pointer
Althara	The primary brand focusing on real estate market analysis, institutional investment, and macro-trends.	`app/brands.py`
Oxono	The secondary brand focusing on technology, systems thinking, and operational efficiency.	`app/brands.py`
Domain	A database-level discriminator (`real_estate` vs `tech`) used to filter news and apply brand-specific logic.	app/models/news.py:26-26
Tone	The linguistic style applied during adaptation (e.g., "analytical/professional" for Althara).	app/adapters/news_adapter.py:4-5

Brand Relationship Diagram

This diagram maps the natural language brand names to their respective code identifiers and domain strings.

```
graph TD
    subgraph "Natural Language Space"
        B1["Althara (Real Estate)"]
```

```
        B2["Oxono (Tech)"]
    end

    subgraph "Code Entity Space"
        C1["domain: 'real_estate'"]
        C2["domain: 'tech'"]
        A1["news_adapter.py"]
        A2["ig_adapter.py"]
        P1["AltharaCategoryV2"]
        P2["constants_tech.py"]
    end

    B1 --- C1
    B2 --- C2
    C1 --> A1
    C1 --> P1
    C2 --> A2
    C2 --> P2
```

Sources: app/brands.py:1-20, app/models/news.py:26-26, app/adapters/news_adapter.py:1-5, app/constants.py:22-59

News Ingestion Terms

Concepts related to the retrieval and initial processing of external content.

- **RSS Ingestor**: The component responsible for fetching XML feeds from configured `RSS_SOURCES` and converting them into `News` ORM objects app/ingestion/rss_ingestor.py:9-51.
- **Guardrails**: A filtering mechanism that uses `DENY_KEYWORDS` and `ALLOW_KEYWORDS` to ensure ingested content is relevant to the brand's domain app/utils/guardrails.py:11-11.
- **Scraping**: The process of visiting a news URL using `httpx` and `BeautifulSoup` to extract the full article text when the RSS summary is insufficient app/ingestion/rss_ingestor.py:56-122.
- **Relevance Score**: An integer (0-100) calculated during ingestion based on keyword matches and category priority, used to rank news in the UI app/ingestion/rss_ingestor.py:138-147.

- **Category Hints**: A dictionary of keywords used to automatically classify a news item into a specific `AltharaCategoryV2` app/ingestion/rss_ingestor.py:130-135.

Ingestion Flow Diagram

```
graph LR
    subgraph "External Sources"
        RSS["RSS Feeds"]
        WEB["Article Webpages"]
    end

    subgraph "rss_ingestor.py"
        FETCH["_extract_article_content"]
        CAT["_categorize_althara_v2"]
        SCORE["_HIGH_VALUE_KEYWORDS"]
    end

    subgraph "Shared Utils"
        GUARD["passes_guardrails"]
        STRIP["strip_html_tags"]
    end

    RSS --> FETCH
    FETCH --> STRIP
    STRIP --> GUARD
    GUARD --> CAT
    CAT --> SCORE
    SCORE --> DB[("SQLAlchemy News Model")]
```

Sources: app/ingestion/rss_ingestor.py:56-147, app/utils/guardrails.py:11-11, app/utils/html_utils.py:9-10

Content Adaptation Terms

Terms related to the transformation of raw news into social-media-ready formats.

- **IG Draft (IGDraft)**: A database record representing a prepared Instagram post, including slides, caption, and hashtags app/models/ig_draft.py:1-30.
- **Carousel Slides**: A JSONB field in the `IGDraft` model containing an array of title/body pairs for Instagram carousels app/models/ig_draft.py:31-31.

- **Althara Content**: A structured JSON object (v2.0 schema) containing "hecho", "lectura", "implicaciones", and "senales_a_vigilar" app/models/news.py:21-21.
- **Strategic Line**: A brand-specific analytical sentence generated based on the news category to provide "expert" context app/adapters/news_adapter.py:121-178.
- **Seed-based Determinism**: The use of a numeric seed (often derived from a timestamp or ID) to select random variations (like "Closers") consistently for a specific record app/adapters/news_adapter.py:180-186.

Data Entity Mapping

This diagram bridges the visual elements of the "News Studio" UI to the underlying SQLAlchemy models.

```
classDiagram
    class NewsStudio_UI {
        +Brand Selector
        +Portal Inbox
        +Draft Editor
    }
    class News_Model {
        +UUID id
        +String title
        +JSONB althara_content
        +Integer relevance_score
    }
    class IGDraft_Model {
        +UUID news_id
        +JSONB carousel_slides
        +Text caption
        +Text status
    }

    NewsStudio_UI --|> News_Model : Displays News
    NewsStudio_UI --|> IGDraft_Model : Edits Drafts
    News_Model "1" --o "*" IGDraft_Model : ig_drafts relationship
```

Sources: app/models/news.py:9-32, app/models/ig_draft.py:1-40, app/templates/draft_editor.html:25-138

Technical Abbreviations

Abbreviation	Meaning	Context
JSONB	Binary JSON	PostgreSQL data type used for `althara_content` and `carousel_slides` app/models/news.py:21-21.
ORM	Object-Relational Mapping	SQLAlchemy usage for `News` and `IGDraft` classes app/database.py.
CTA	Call to Action	The closing line of a social post (e.g., "Guárdalo") app/adapters/ig_adapter.py:83-84.
BOE	Boletín Oficial del Estado	Source for auction/legal news (`BOE_SUBASTAS`) app/constants.py:47-47.
SOCIMI	Sociedades Anónimas Cotizadas de Inversión en el Mercado Inmobiliario	Specific real estate investment vehicle referenced in `INVERSION_INSTITUCIONAL` app/constants.py:40-40.

Sources: app/models/news.py:1-32, app/constants.py:40-116, app/adapters/ig_adapter.py:83-122